GEMPRO Notebook

GEMPRO

Contest

Os resultados e intermediários devem caber nos tipos usados.

template.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); i++)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.vimrc

```
set cin ts=4 sw=4 udf cul nu is

ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
↪  md5sum \| cut -c -6
```

.bashrc

```
c() {
    g++ $1.cpp -o $1 -std=c++20 \
        -Wall -Wextra -Wshadow -Wconversion -Wpedantic
}
cs() {
    g++ $1.cpp -o $1 -std=c++20 \
        -Wall -Wextra -Wshadow -Wconversion -Wpedantic \
        -fsanitize=address,undefined \
        -fsanitize=signed-integer-overflow -ggdb
}
gen() {
    local N=${1:-10}
    for i in $(seq 1 $N); do
        python3 gen.py $i > $2$i.in
    done
}
rr() {
    local ext=${2:-out}
    for i in $(ls -v *.in); do
        local f=$(basename $i .in)
        ./$1 < $f.in > $f.$ext
    done
}
chk() {
    local a=${1:-ans}
    local b=${2:-out}
    for i in $(ls -v *.$a); do
        local f=$(basename $i .$a)
        echo -n "$f.$a - $f.$b > "
        if cmp -s $f.$a $f.$b; then
            echo OK
        else
            echo X
        fi
    done
}
```

Template de stress testing

```
from random import randrange as rr, shuffle as sh, sample as
↪  sm, choice as ch, seed
from string import ascii_lowercase as alpha
from itertools import combinations as combs, permutations as
↪  perms, product as prod
from sys import argv

if len(argv) > 1: seed(argv[1])
```

Matemática

Algoritmo euclideano. `g = egcd(a,b,x,y)`, $ax + by = g$. $O(\log(\min(a, b)))$.

b4f64d - math/egcd.cpp - 9 lines

```cpp
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

CRT. `crt(r1,m1,r2,m2)`: `{r,m}`, $0 \le r < m$, ou `{0,-1}` se incompatível. Módulos positivos, resíduos normalizados. $O(\log \min(m_1, m_2))$.

6e0e1c - math/crt.cpp - 9 lines

```cpp
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Teste de primalidade. `isPrime(n)` para inteiros de 64 bits em $O(\log n)$.

cabafb - math/primality.cpp - 25 lines

```cpp
using i128 = __int128;

bool checkPrime(ll p, ll a) {
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p < 2) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    ↪  {
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // 2c12c6
```

Fatoração de inteiros. `factor(n,fs)` acrescenta fatores primos com multiplicidade, em ordem arbitrária. Esperado $O(\sqrt[4]{n})$.

7b1297 - math/factor.cpp - 28 lines

```cpp
mt19937_64 rng64(random_device{}());

using ull = unsigned long long;
using u128 = __uint128_t;

ull modMul(ull a, ull b, ull mod) { return (u128)a * b % mod;
↪ }

ull rho(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ull c = uniform_int_distribution<ull>(1, n - 1)(rng64);
    ull x = uniform_int_distribution<ull>(0, n - 1)(rng64), y
    ↪  = x, d = 1;
    auto f = [&](ull x) { return (modMul(x, x, n) + c) % n; };
    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? rho(n) : d;
} // 8e1254

void factor(ull n, vector<ull> &fs) {
    if (n == 1) return;
    if (isPrime(n)) { fs.pb(n); return; }
    ull d = rho(n);
    factor(d, fs);
    factor(n / d, fs);
} // 6d58e9
```

Raiz quadrada módulo `mod`. `sqrtMod(a)`: menor raiz não negativa, ou -1. Requer `mod` primo e $0 \le a < \texttt{mod}$. $O(\log^2 \texttt{mod})$.

b04743 - math/mod-sqrt.cpp - 19 lines

```cpp
int sqrtMod(int a) { // mod prime
    if (a == 0 || mod == 2) return a;
    if (powm(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return powm(a, (mod + 1) / 4);
    int s = __builtin_ctz(mod - 1), q = (mod - 1) >> s, z = 2;
    while (powm(z, (mod - 1) / 2) != mod - 1) z++;
    ll c = powm(z, q), x = powm(a, (q + 1) / 2), t = powm(a,
    ↪  q);
    for (int m = s; t != 1; ) {
        ll tt = t;
        int i = 0;
        while (tt != 1) tt = tt * tt % mod, i++;
        ll b = powm(c, 1 << (m - i - 1));
        x = x * b % mod;
        c = b * b % mod;
        t = t * c % mod;
        m = i;
    }
    return min<ll>(x, mod - x);
} // b04743
```

Soma de floor. `floorSums(n,m,a,b)` retorna $\{\sum f_i, \sum i f_i, \sum f_i^2\}$ em `i128`, $f_i = \lfloor (ai+b)/m \rfloor$, $0 \le i < n$. `floorSum` calcula só a primeira. Requer $n, a, b \ge 0$, $m > 0$. Custo logarítmico.

9744fc - math/floor-sum.cpp - 24 lines

```cpp
using i128 = __int128_t;
array<i128,3> floorSums(ll n, ll m, ll a, ll b) {
    if (!n) return {0,0,0};
    i128 s1 = (i128)n*(n-1)/2, s2 =
    ↪  (i128)n*(n-1)*(2*(i128)n-1)/6;
    ll qa = a/m, qb = b/m;
    if (qa || qb) {
        auto [x,y,z] = floorSums(n,m,a%m,b%m);
        return {x + qa*s1 + (i128)qb*n, y + qa*s2 + qb*s1,
            z + 2*qa*y + 2*qb*x + (i128)qa*qa*s2
            + 2*(i128)qa*qb*s1 + (i128)qb*qb*n};
    }
    ll k = ((i128)a*(n-1)+b)/m;
    if (!k) return {0,0,0};
    auto [x,y,z] = floorSums(k,a,m,m-b-1);
    return {(i128)k*(n-1)-x, k*s1-(z+x)/2,
    ↪  (i128)k*k*(n-1)-2*y-x};
} // 164b59
i128 floorSum(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    if (a >= m || b >= m)
        return (i128)(a/m)*n*(n-1)/2 + (i128)(b/m)*n
            + floorSum(n,m,a%m,b%m);
    ll k = ((i128)a*(n-1)+b)/m;
    return k ? (i128)k*(n-1)-floorSum(k,a,m,m-b-1) : 0;
} // 3d0695
```

Eliminação gaussiana. `gauss(a,det)`: escalona **in-place**, retorna posto e escreve determinante módulo `mod` (0 se retangular; 1 para 0×0). Módulo primo, entradas normalizadas; $O(nm \min(n,m) + \min(n,m) \log \texttt{mod})$. `linearSystem(a,m,x,basis)` recebe matriz aumentada $n \times (m+1)$, sem modificá-la; retorna falso se inconsistente. Caso contrário, soluções $x + \sum c_i \, \texttt{basis}[i]$. Aceita $n = 0$ e $m = 0$; saída ocupa $O(m^2)$.

5dcc86 - math/gaussian-elimination.cpp - 44 lines

```cpp
int gauss(vector<vector<ll>> &a, ll &det) {
    int n = sz(a), m = n ? sz(a[0]) : 0, r = 0;
    det = 1;
    rep(c,0,m) {
        int p = r;
        while (p < n && !a[p][c]) p++;
        if (p == n) continue;
        if (p != r) det = (mod - det) % mod;
        swap(a[p], a[r]);
        ll pivot = (a[r][c] + mod) % mod;
        det = det * pivot % mod;
        ll iv = powm(pivot, mod - 2);
        rep(j,c,m) a[r][j] = a[r][j] * iv % mod;
        rep(i,0,n) if (i != r && a[i][c]) {
            ll x = a[i][c];
            rep(j,c,m) a[i][j] = (a[i][j] - x * a[r][j]) %
            ↪  mod;
        }
        r++;
    }
    rep(i,0,n) rep(j,0,m) if (a[i][j] < 0) a[i][j] += mod;
    if (n != m || r < n) det = 0;
    return r;
} // 14ff56
bool linearSystem(vector<vector<ll>> a, int m,
        vector<ll> &x, vector<vector<ll>> &basis) {
    ll det;
    gauss(a,det);
    vi row(m,-1);
    rep(i,0,sz(a)) {
        int j = 0;
        while (j <= m && !a[i][j]) j++;
        if (j == m) return false;
        if (j < m) row[j] = i;
    }
    x.assign(m,0); basis.clear();
    rep(j,0,m) if (row[j] >= 0) x[j] = a[row[j]][m];
    rep(f,0,m) if (row[f] < 0) {
        vector<ll> v(m); v[f] = 1;
        rep(j,0,m) if (row[j] >= 0)
            v[j] = (mod-a[row[j]][f])%mod;
        basis.pb(v);
    }
    return true;
} // dbc0b6
```

Base de XOR. `XorBasis<B=63>`: `add(x)` retorna 1 se dependente (forma subconjunto não vazio de XOR zero com anteriores); senão, insere e retorna 0. `query(x=0,hi=0)` minimiza $x \oplus$ XOR de um subconjunto; `hi` $= 1$ maximiza.

`w=reduce(x)` minimiza x in-place; se zerar, `w` reconstrói o original. Bit j seleciona a j-ésima entrada independente, em ordem de inserção (0-based). $O(B\lceil B/64\rceil)$ por operação.

992a74 - math/xor-basis.cpp - 22 lines

```cpp
template<int B = 63>
struct XorBasis {
    using T = bitset<B>;
    struct Row { int i; T x,w; };
    vector<Row> b;
    T reduce(T &x, bool hi = 0) const {
        T w;
        for (auto &r : b) if (x[r.i] != hi) x ^= r.x, w ^= r.w;
        return w;
    } // 38829e
    bool add(T x) {
        auto w = reduce(x);
        for (int i = B-1; i >= 0; --i) if (x[i]) {
            w[sz(b)] = 1;
            int j = 0;
            while (j < sz(b) && b[j].i > i) ++j;
            b.insert(b.begin()+j,{i,x,w}); return 0;
        }
        return 1;
    } // a3661d
    T query(T x = {}, bool hi = 0) const { reduce(x,hi); return x; }
};
```

Interpolação de Lagrange. `lagrange(xs,ys,x)` avalia o interpolador: $O(n^2 + n \log \texttt{mod})$. `lagrange(ys,x)` usa abscissas $0, \ldots, n-1$: $O(n + \log \texttt{mod})$. Requer módulo primo, resíduos normalizados e abscissas distintas módulo `mod`; na segunda, $1 \le n \le \texttt{mod}$.

29e498 - math/lagrangian-interpolation.cpp - 26 lines

```cpp
ll lagrange(const vector<ll> &xs, const vector<ll> &ys, ll x) {
    int n = sz(xs); ll ans = 0;
    rep(i,0,n) if (xs[i] == x) return ys[i];
    rep(i,0,n) {
        ll a = ys[i], b = 1;
        rep(j,0,n) if (i != j) a = a * (x - xs[j] + mod) % mod, b = b * (xs[i] - xs[j] + mod) % mod;
        ans = (ans + a * powm(b, mod - 2)) % mod;
    }
    return ans;
} // 51ae24

ll lagrange(const vector<ll> &y, ll x) {
    int n = sz(y);
    x = (x % mod + mod) % mod;
    if (x < n) return y[x];
    vector<ll> p(n + 1, 1), s(n + 1, 1), a(n);
    rep(i,0,n) p[i + 1] = p[i] * (x - i + mod) % mod;
    for (int i = n - 1; i >= 0; --i) s[i] = s[i + 1] * (x - i + mod) % mod;
    a[0] = 1;
    rep(i,1,n) a[i] = a[i - 1] * i % mod;
    a[n - 1] = powm(a[n - 1], mod - 2);
    for (int i = n - 1; i; --i) a[i - 1] = a[i] * i % mod;
    ll ans = 0;
    rep(i,0,n) ans = (ans + p[i] * s[i + 1] % mod * a[i] % mod * a[n - 1 - i] % mod * (((n - 1 - i) & 1) ? mod - y[i] : y[i])) % mod;
    return ans;
} // 05868c
```

Polinômio de Lagrange. `lagrangePoly(xs,ys)`: coeficientes por grau crescente. Requer $n \ge 1$, módulo primo, resíduos normalizados e abscissas distintas módulo `mod`. $O(n^2 + n \log \texttt{mod})$.

7d0710 - math/lagrangian-polynomial.cpp - 19 lines

```cpp
vector<ll> lagrangePoly(const vector<ll> &xs, const vector<ll> &ys) {
    int n = sz(xs);
    vector<ll> p = {1}, f(n);
    rep(i,0,n) {
        p.pb(0);
        for (int j = i + 1; j; --j) p[j] = (p[j - 1] + mod - p[j] * xs[i] % mod) % mod;
        p[0] = p[0] * (mod - xs[i]) % mod;
    }
    rep(i,0,n) {
        vector<ll> q(n);
        q[n - 1] = p[n];
        for (int j = n - 1; j; --j) q[j - 1] = (p[j] + q[j] * xs[i]) % mod;
        ll den = 1;
        rep(j,0,n) if (i != j) den = den * (xs[i] - xs[j] + mod) % mod;
        ll c = ys[i] * powm(den, mod - 2) % mod;
        rep(j,0,n) f[j] = (f[j] + c * q[j]) % mod;
    }
    return f;
} // 7d0710
```

Berlekamp-Massey. `berlekampMassey(s)`: coeficientes c da menor recorrência compatível, $s_i = \sum_{j=0}^{m-1} c_j s_{i-j-1}$. Módulo primo, termos normalizados; sequência toda zero retorna vazio. Para recuperar ordem m, forneça ao menos $2m$ termos. $O(n^2 + n \log \texttt{mod})$.

68a616 - math/berlekamp-massey.cpp - 19 lines

```cpp
vi berlekampMassey(vi s) {
    vi c(1, 1), b(1, 1);
    int l = 0, m = 1; ll bb = 1;
    rep (n, 0, sz(s)) {
        ll d = 0;
        rep (i, 0, l + 1) d = (d + 1LL * c[i] * s[n - i]) % mod;
        if (d < 0) d += mod;
        if (!d) { m++; continue; }
        vi t = c;
        ll x = d * powm(bb, mod - 2) % mod;
        if (sz(c) < sz(b) + m) c.resize(sz(b) + m);
        rep (i, 0, sz(b)) c[i + m] = (c[i + m] - x * b[i]) % mod;
        if (2 * l <= n) l = n + 1 - l, b = t, bb = d, m = 1;
        else m++;
    }
    c.erase(begin(c));
    for (int &x : c) x = (mod - x) % mod;
    return c;
} // 68a616
```

Recorrência linear. `linRec(s,tr,k)`: termo $k \ge 0$, 0-based; `s` contém os primeiros $m = |\texttt{tr}| > 0$ termos e $s_i = \sum_{j=0}^{m-1} \texttt{tr}_j s_{i-j-1}$. $O(m^2 \log(k+1))$.

5f7ef4 - math/linear-recurrence.cpp - 23 lines

```cpp
int linRec(vi s, vi tr, ll k) { // s = first m terms, 0-indexed kth term
    int n = sz(tr);
    if (k < n) return s[k];
    auto mul = [&](vi a, vi b) {
        vi c(2 * n + 1);
        rep (i, 0, n + 1) rep (j, 0, n + 1)
            c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep (j, 0, n)
            c[i - 1 - j] = (c[i - 1 - j] + 1LL * c[i] * tr[j]) % mod;
        c.resize(n + 1);
        return c;
    }; // 610f7a
    vi pol(n + 1), e(n + 1);
    pol[0] = 1;
    e[1] = 1;
    for (++k; k; k >>= 1) {
        if (k & 1) pol = mul(pol, e);
        e = mul(e, e);
    }
    ll ans = 0;
    rep (i, 0, n) ans = (ans + 1LL * pol[i + 1] * s[i]) % mod;
```

```
    return ans;
} // 5f7ef4
```

Bostan–Mori. `bostanMori(p,q,k)`: $[x^k]P/Q$, $k \geq 0$, $Q(0) \neq 0$, $\deg P < \deg Q$. `fastRec(s,c,k)`: $d = |c| > 0$ termos iniciais, $s_i = \sum_{j=0}^{d-1} c_j s_{i-j-1}$. Coeficientes normalizados; convoluções devem caber no limite da transformada. $O(d \log d \log(k+1))$ mais convolução inicial.

8f1b17 - math/bostan-mori.cpp - 20 lines

```
ll bostanMori(vector<ll> p, vector<ll> q, ll k) {
    while (k) {
        auto neg = q;
        for (int i = 1; i < sz(q); i += 2) neg[i] = (mod -
        ↪  neg[i]) % mod;
        auto a = conv(p,neg), b = conv(q,neg);
        p.clear(), q.clear();
        for (int i = k & 1; i < sz(a); i += 2) p.pb(a[i]);
        for (int i = 0; i < sz(b); i += 2) q.pb(b[i]);
        k >>= 1;
    }
    return p.empty() ? 0 : p[0] * powm(q[0],mod-2) % mod;
} // a87071
ll fastRec(vector<ll> s, vector<ll> c, ll k) {
    int d = sz(c);
    vector<ll> q(d+1,1);
    rep(i,0,d) q[i+1] = (mod-c[i]) % mod;
    auto p = conv(s,q);
    p.resize(d);
    return bostanMori(p,q,k);
} // 37d63c
```

Estruturas de Dados

Subset sum. `subSum(x)`: $x_i \geq 0$, $S = \sum x_i$. Retorna `p[s]` = índice usado para atingir s, ou -1; soma 0 é possível apesar de `p[0]=-1`. Reconstrua com $s \leftarrow s - x_{p[s]}$ até 0. $O(n(1 + S/64) + S)$.

a4f267 - dsa/fast-subset-sum.cpp - 22 lines

```
vi subSum(vi x) {
    using ull = unsigned long long;
    int s = reduce(all(x));
    vector<ull> bs(s / 64 + 1);
    bs[0] = 1;
    vi p(s + 1, -1);
    rep(i,0,sz(x)) {
        int a = x[i], q = a / 64, r = a % 64;
        for (int j = sz(bs) - 1; j >= 0; --j) {
            ull pre = bs[j], add = 0;
            if (j >= q) add |= (q ? bs[j - q] : pre) << r;
            if (r && j > q) add |= bs[j - q - 1] >> (64 - r);
            bs[j] |= add;
            for (ull k = bs[j] ^ pre; k; k &= k - 1) {
                int t = __builtin_ctzll(k);
                int v = 64 * j + t;
                if (v <= s) p[v] = i;
            }
        }
    }
    return p;
} // a4f267
```

Sparse table. `build(v)`, `query(l,r)` em $[l, r)$ não vazio. `op` associativa e idempotente. Construção $O(n \log n)$; consulta $O(1)$.

9310ff - dsa/sparse-table.cpp - 15 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
    void build(vector<S> &v) {
        int n = sz(v), levels = max(1,
        ↪  (int)bit_width((unsigned)n));
        t.assign(levels, vector<S>(n));
        t[0] = v;
        rep(k,1,levels)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k-1][i], t[k-1][i+(1 <<
                ↪  (k-1))]);
    } // 891c13
    S query(int l, int r) {
        int k = bit_width((unsigned)(r-l))-1;
        return op(t[k][l], t[k][r-(1 << k)]);
    } // 0a29bf
};
```

Segtree. `set(i,x)`, `query(l,r)` em $[l, r)$; monoide `op,e`, inclusive não comutativo. Índices 0-based. $O(\log n)$ por operação.

b76e87 - dsa/segtree.cpp - 18 lines

```
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i <<
        ↪  1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy segtree. `set(i,x)`, `apply(l,r,u)`, `sum(l,r)` em $[l, r)$, $n > 0$. `compose(novo,antigo)` aplica antigo antes de novo. `mapping` deve respeitar `op`; $O(\log n)$ por operação.

21bbc4 - dsa/lazy-segtree.cpp - 50 lines

```
template<class S, S (*op)(S, S), S (*e)(), class U, S
↪  (*mapping)(U, S),
        U (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(4 * N, id()), n(N) {}
    void applyNode(int i, U u) {
        t[i] = mapping(u, t[i]);
        d[i] = compose(u, d[i]);
    } // 3e5d0d
    void push(int i) {
        applyNode(i << 1, d[i]);
        applyNode(i << 1 | 1, d[i]);
        d[i] = id();
    } // 7836a8
    void pull(int i) { t[i] = op(t[i << 1], t[i << 1 | 1]); }
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        if (tl + 1 == tr) {
            t[i] = val;
            d[i] = id();
            return;
        }
        push(i);
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
        else set(j, val, tm, tr, i << 1 | 1);
        pull(i);
    } // f9c2fd
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) return applyNode(i, u);
        push(i);
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
        pull(i);
    } // 3f0fb7
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
```

```
    S sum(int l, int r, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
        push(i);
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1),
                  sum(l, r, tm, tr, i << 1 | 1));
    } // b976f6
};
```

Consultas de Mo. `moOrd(q,n)`: permutação dos índices das consultas `{l,r}`. Bloco $\sqrt{n}$: $O(Q \log Q)$ para ordenar e $O((n+Q)\sqrt{n})$ movimentos, vezes o custo de adicionar/remover.

c2c90b - dsa/mo-queries.cpp - 11 lines

```
vi moOrd(vector<pii> &q, int n) {
    int B = max(1, (int)sqrt(n));
    vi o(sz(q));
    iota(all(o), 0);
    sort(all(o), [&](int i, int j) {
        int a = q[i].fi / B, b = q[j].fi / B;
        if (a != b) return a < b;
        return (a & 1) ? q[i].se > q[j].se : q[i].se <
        ↪ q[j].se;
    } /* f3c775 */);
    return o;
} // c2c90b
```

Wavelet matrix. `wavMat(a)`: $0 \le a_i < 2^H$. Consultas em $[l,r)$ com altura inicial H: `rangeKth(...,k)`, $0 \le k < r-l$; `cntLess(...,x)`, $0 \le x < 2^H$. Construção $O(Hn)$; consulta $O(H)$.

399393 - dsa/wavelet-matrix.cpp - 36 lines

```
const int H = 31;
using WM = array<vi, H>;
WM wavMat(vector<ll> a) {
    WM wm;
    int n = sz(a);
    for (int i = H - 1; i >= 0; i--) {
        wm[i].resize(n + 1);
        rep(j, 1, n + 1) wm[i][j] = a[j - 1] >> i & 1;
        stable_partition(all(a), [&](ll x) { return ~x >> i &
        ↪ 1; });
        rep(j, 1, n + 1) wm[i][j] += wm[i][j - 1];
    }
    return wm;
} // 9aeaa7

tuple<int, int, int, int> getSplit(WM &wm, int h, int l, int
↪ r) {
    int a1 = wm[h][l], a0 = l - a1,
        b1 = wm[h][r], b0 = r - b1,
        n = sz(wm[0]) - 1, c = n - wm[h][n];
    return {a0, b0, c + a1, c + b1};
} // f64ad6

ll rangeKth(WM &wm, int h, int l, int r, int k) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    int sl = r0 - l0;
    if (k < sl) return rangeKth(wm, h - 1, l0, r0, k);
    return (1 << (h - 1)) + rangeKth(wm, h - 1, l1, r1, k -
    ↪ sl);
} // 97e884

int cntLess(WM &wm, int h, int l, int r, ll x) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    if (x >> (h - 1) & 1)
        return r0 - l0 + cntLess(wm, h - 1, l1, r1, x);
    return cntLess(wm, h - 1, l0, r0, x);
} // fcb25d
```

Wavelet matrix: posições e retângulos. `wavPos(wm)` retorna `pos[h][i]`: posição do índice original i após a partição do nível h, em $O(Hn)$. `rectOp(wm,l,r,a,b,f)` cobre $i \in [l,r)$, $a \le x_i < b$ por $O(H)$ chamadas disjuntas `f(h,L,R)` na ordem de `pos[h]`. Requer $0 \le a \le b \le 2^H$.

445682 - dsa/wavelet-matrix-extended.cpp - 26 lines

```
WM wavPos(WM &wm) {
    WM pos;
    int n = sz(wm[0])-1;
    vi ord(n); iota(all(ord),0);
    for (int h = H-1; h >= 0; --h) {
        pos[h].resize(n);
        rep(i,0,n) {
            int k = ord[i], ones = wm[h][k];
            ord[i] = wm[h][k+1] > ones ? n-wm[h][n]+ones :
            ↪ k-ones;
            pos[h][i] = ord[i];
        }
    }
    return pos;
} // d118d3

void rectOp(WM &wm, int l, int r, ll a, ll b, auto f) {
    auto go = [&](auto &&go, int h, int l, int r, ll lo, ll
    ↪ hi) -> void {
        if (l == r || b <= lo || hi <= a || a == b) return;
        if (h < H && a <= lo && hi <= b) { f(h,l,r); return; }
        auto [l0,r0,l1,r1] = getSplit(wm,h-1,l,r);
        ll mid = (lo+hi)/2;
        go(go,h-1,l0,r0,lo,mid);
        go(go,h-1,l1,r1,mid,hi);
    }; // 305538
    go(go,H,l,r,0,1LL<<H);
} // 51d66e
```

Árvore cartesiana. `cartTree(n,le)`: pai de cada índice; a raiz é seu próprio pai. `le(i,j)` decide se i fica acima de j, inclusive desempate. $O(n)$.

caeaba - dsa/cartesian-tree.cpp - 20 lines

```
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.pb(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // caeaba
```

Treap implícita. `split(t,k,a,b)` separa os primeiros k elementos; `sep(t,l,r,a,b,c)` isola $[l,r)$ em `b`. Índices 0-based em `ins,era,kth`. $O(\log n)$ esperado.

ad716f - dsa/treap.cpp - 69 lines

```
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};

int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }
```

```cpp
void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b),
    ↪ pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488
void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Extensões GNU. Requer libstdc++. Árvore: $O(\log n)$, comparador estrito; repetidos usam `{valor,id}`. Hash: $O(1)$ esperado amortizado; rehash invalida iteradores/referências. Heap: `push/join` $O(1)$, demais operações $O(\log n)$ amortizado; handles sobrevivem a `join`. Rope preserva cópias; `substr/erase(pos,len)` usam quantidade. Acesso $O(h)$; edições podem ser lineares. Bitsets: buscas retornam `size()` se ausente; `clear()` do dinâmico reduz tamanho a zero, `reset()` apenas zera bits.

```cpp
#include <functional>
#include <utility>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/priority_queue.hpp>
namespace pb = __gnu_pbds;

template<class K, class V = pb::null_type, class C =
↪ std::less<K>>
using OrderedTree = pb::tree<K,V,C,pb::rb_tree_tag,
    pb::tree_order_statistics_node_update>;
template<class K, class V = pb::null_type, class H =
↪ std::hash<K>>
using HashTable = pb::gp_hash_table<K,V,H>;
template<class T, class C = std::less<T>>
using Heap = pb::priority_queue<T,C,pb::pairing_heap_tag>;

void pbdsExample() {
    OrderedTree<std::pair<int,int>> s; // {valor,id}, id >= 0
    s.insert({5,0}); s.insert({5,1});
    s.order_of_key({5,-1}); // 0: quantidade < 5
    s.order_of_key({6,-1}); // 2: quantidade < 6
    auto it = s.find_by_order(1); // {5,1}; end() se k >=
    ↪ size()
    s.erase(it); // remove apenas esta ocorrencia
    OrderedTree<int,int> m; m[4] = 9; // mapa com rank por
    ↪ chave
    HashTable<int,int> h; h[3] = 7;
    HashTable<int> seen; seen.insert(3);
    Heap<int> a, b; // max-heap; std::greater<int> para
    ↪ minimo
    auto p = a.push(4); // Heap<int>::point_iterator
    a.modify(p,1);
    auto q = b.push(2);
    a.join(b); // transfere b para a; b fica vazio
    a.erase(q); // handle de b agora pertence a a
    a.top(); // 1
    a.pop(); // invalida p
}

#include <ext/rope>
using __gnu_cxx::rope;
using __gnu_cxx::crope; // rope<char>

void ropeExample() {
    int v[] = {10,20,30};
    rope<int> a(v,v+3);
    auto old = a; // snapshot
    auto part = a.substr(1,2);
    a.erase(1,2); // posicao, quantidade
    a.insert(0,part); // [20,30,10]; old = [10,20,30]
    a.replace(0,99); // [99,30,10]; part = [20,30]
    a[1]; // 30
    crope s("abc"); s += "def";
}

#include <bitset>
#include <tr2/dynamic_bitset>

void bitsetExample(int n, int x) { // n > 0, x >= 0
    std::bitset<100> b;
    b.set(3); b.set(70);
    for (size_t i = b._Find_first(); i < b.size(); i =
    ↪ b._Find_next(i)) {
        // processar i
    }
    std::tr2::dynamic_bitset<> d(n);
    d[0] = 1; d |= d << x; // subset sum
    for (size_t i = d.find_first(); i < d.size(); i =
    ↪ d.find_next(i)) {
        // processar i
    }
    d.resize(n+1); // preserva bits; novos bits = 0
}
```

Árvore de Li Chao. `LiChao(N)` mantém o mínimo em $[0, N)$, $N > 0$. `insert(line)`: reta inteira, $O(\log N)$; `insert(line,L,R)`: ativa em $[L, R)$, $O(\log^2 N)$. `query(x)` retorna a melhor reta em $O(\log N)$. Para máximo, inverta comparações e sentinela.

a7d6fc - dsa/li-chao-extended.cpp - 35 lines

```cpp
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
```

```
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {}
    void insert(Line line) { insert(line, 0, n); }
    void insert(Line line, int L, int R) { insert(line, L, R,
    ↪ 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int
    ↪ i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]);
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm,
            ↪ tr, i << 1 | 1);
            else insert(line, L, R, tl, tm, i << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 3dcc38
    Line query(int x) {
        Line line = {0, LLONG_MAX};
        for (int tl = 0, tr = n, i = 1;;) {
            if (t[i](x) < line(x)) line = t[i];
            if (tl + 1 == tr) return line;
            int tm = tl + (tr-tl)/2;
            if (x < tm) tr = tm, i <<= 1;
            else tl = tm, i = i << 1 | 1;
        }
    } // cc25f8
};
```

Simplex. `Simplex(A,b,c).solve(x)` maximiza $c \cdot x$ com $Ax \leq b$, $x \geq 0$. Escreve uma solução em `x`; retorna $-$INF se inviável, $+$INF se ilimitado.

becd4c - dsa/simplex.cpp - 57 lines

```
struct Simplex {
    static constexpr ld EPS = 1e-10;
    static constexpr ld INF = 1e100;
    int m, n;
    vi B, N;
    vector<vector<ld>> D;
    Simplex(vector<vector<ld>> A, vector<ld> b, vector<ld> c)
        : m(sz(b)), n(sz(c)), B(m), N(n + 1), D(m + 2,
        ↪ vector<ld>(n + 2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] =
        ↪ b[i];
        rep(j,0,n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        rep(i,0,m+2) if (i != r)
            rep(j,0,n+2) if (j != s) D[i][j] -= D[r][j] *
            ↪ D[i][s] * inv;
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    } // eb9c6e
    bool simplex(int ph) {
        int x = ph == 1 ? m + 1 : m;
        while (1) {
            int s = -1;
            rep(j,0,n+1) if (!(ph == 2 && N[j] == -1) &&
                (s == -1 || D[x][j] < D[x][s] - EPS ||
                (abs(D[x][j] - D[x][s]) <= EPS && N[j] <
                ↪ N[s]))) s = j;
            if (D[x][s] >= -EPS) return 1;
            int r = -1;
            rep(i,0,m) if (D[i][s] > EPS &&
                (r == -1 || D[i][n + 1] / D[i][s] < D[r][n +
                ↪ 1] / D[r][s] - EPS ||
                (abs(D[i][n + 1] / D[i][s] - D[r][n + 1] /
                ↪ D[r][s]) <= EPS && B[i] < B[r]))) r = i;
            if (r == -1) return 0;
            pivot(r, s);
        }
    } // fee585
    ld solve(vector<ld> &x) {
        int r = 0;
        rep(i,1,m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            pivot(r, n);
            if (!simplex(1) || D[m + 1][n + 1] < -EPS) return
            ↪ -INF;
            rep(i,0,m) if (B[i] == -1) {
                int s = -1;
                rep(j,0,n+1) if (s == -1 || D[i][j] < D[i][s]
                ↪ - EPS ||
                    (abs(D[i][j] - D[i][s]) <= EPS && N[j] <
                    ↪ N[s])) s = j;
                pivot(i, s);
            }
        }
        if (!simplex(2)) return INF;
        x.assign(n, 0);
        rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return D[m][n + 1];
    } // 38c01f
};
```

Convolução

$c_k = \sum_{i+j=k} a_i b_j$; $O(n \log n)$.

FFT

e42ddb - dsa/fft.cpp - 41 lines

```
using cd = complex<ld>;
const ld PI = acosl(-1.0L);

void fft(vector<cd> &a) {
    int n = sz(a);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        rt.resize(n);
        cd x = polar((ld)1, PI / k);
        rep(i, k, 2 * k) rt[i] = i & 1 ? rt[i / 2] * x : rt[i
        ↪ / 2];
    }
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            rep(j, 0, k) {
                cd z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // 29b822

vector<ld> conv(const vector<ld> &a, const vector<ld> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    vector<cd> in(n), out(n);
    rep(i, 0, sz(a)) in[i].real(a[i]);
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (cd &x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    vector<ld> res(s);
    rep(i, 0, s) res[i] = imag(out[i]) / (4 * n);
    return res;
} // 312a10
```

NTT

`conv(a,b)`: coeficientes normalizados módulo `mod`. O tamanho arredondado para potência de 2 deve ser $\leq 2^{23}$ nesta implementação; `ntt` modifica o vetor e exige tamanho potên-

cia de 2.

bec19a - dsa/ntt.cpp - 38 lines

```cpp
// const int mod = 998244353, ROOT = 3;
// could also be mod = 469762049, ROOT = 3
void ntt(vector<ll> &a) {
    int n = sz(a);
    static vector<ll> r = {1, 1};
    for (int t = sz(r); t < n; t <<= 1) {
        r.resize(t << 1);
        ll z = powm(ROOT, (mod - 1) / (t << 1));
        for (int i = t; i < t << 1; i++)
            r[i] = r[i >> 1] * (i & 1 ? z : 1) % mod;
    }
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int m = 1; m < n; m <<= 1)
        for (int l = 0; l < n; l += m << 1)
            rep(i, 0, m) {
                ll x = a[l + i], z = r[m + i] * a[l + m + i] %
                ↪ mod;
                a[l + i] = (x + z < mod ? x + z : x + z -
                ↪ mod);
                a[l + m + i] = (x < z ? x - z + mod : x - z);
            }
} // 3904ec

vector<ll> conv(vector<ll> &a, vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    int inv = powm(n, mod - 2);
    vector<ll> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n) out[-i & (n - 1)] = L[i] * R[i] % mod * inv %
    ↪ mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // ad22f1
```

Convolução de XOR. `xorConv(a,b)`: $c_k = \sum_{i \oplus j = k} a_i b_j$; $O(n \log n)$.

fd3fd7 - dsa/xor-convolution.cpp - 20 lines

```cpp
void fwht(vector<ll> &a) {
    for (int k = 1; k < sz(a); k <<= 1)
        for (int i = 0; i < sz(a); i += 2 * k)
            rep (j, i, i + k) {
                ll x = a[j], y = a[j + k];
                a[j] = x + y; // mod: a[j] = (x + y) % mod;
                a[j + k] = x - y; // mod: a[j + k] = (x - y +
                ↪ mod) % mod;
            }
} // ae23e1

vector<ll> xorConv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < max(sz(a), sz(b))) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    rep (i, 0, n) a[i] *= b[i]; // mod: a[i] = a[i] * b[i] %
    ↪ mod;
    fwht(a);
    rep (i, 0, n) a[i] /= n; // mod: a[i] = a[i] * invN %
    ↪ mod;
    return a;
} // 0bc2c7
```

Zeta e Mobius. `subset/superset`: máscaras em $[0, 2^B)$, $O(B2^B)$. `div/mul`: divisores/múltiplos em $[1, n]$ (vetor de tamanho $n+1$; índice 0 ignorado). Passe `inv=true` para a inversa de Möbius; evite overflow em `int`.

1abb33 - dsa/zeta.cpp - 38 lines

```cpp
vi subsetZeta(vi a, bool inv = false) {
    for (int j = 1; j < sz(a); j <<= 1)
        rep(i,1,sz(a)) if (i & j) inv ? a[i] -= a[i^j] : a[i]
        ↪ += a[i^j];
    return a;
} // 2e2d58
vi supersetZeta(vi a, bool inv = false) {
    for (int j = 1; j < sz(a); j <<= 1)
        rep(i,1,sz(a)) if (i & j) inv ? a[i^j] -= a[i] :
        ↪ a[i^j] += a[i];
    return a;
} // c2aec0
vi enumPrimes(int n) {
    vi pr, spf(n);
    rep(i, 2, n) {
        if (spf[i] == 0) spf[i] = i, pr.pb(i);
        for (int j: pr) {
            if (i * j >= n) break;
            spf[i * j] = j;
            if (j == spf[i]) break;
        }
    }
    return pr;
} // 46cddf
vi divZeta(vi a, bool inv = false) {
    int n = sz(a)-1;
    for (int p : enumPrimes(n+1)) rep(j,1,n/p+1) {
        int i = inv ? n/p+1-j : j;
        inv ? a[i*p] -= a[i] : a[i*p] += a[i];
    }
    return a;
} // 2db431
vi mulZeta(vi a, bool inv = false) {
    int n = sz(a)-1;
    for (int p : enumPrimes(n+1)) rep(j,1,n/p+1) {
        int i = inv ? j : n/p+1-j;
        inv ? a[i] -= a[i*p] : a[i] += a[i*p];
    }
    return a;
} // 486237
```

Convolução de subconjuntos. Convolução de OR para subconjuntos disjuntos. $O(n \log(n)^2)$.

a32e26 - dsa/subset-convolution.cpp - 16 lines

```cpp
#define T popcount(unsigned(i))
vi subsetConv(vi &a, vi &b) {
    int n = sz(a), m = bit_width(unsigned(n));
    vector ca(m, vi(n)), cb(m, vi(n)), c(m, vi(n));
    rep(i, 0, n) ca[T][i] = a[i], cb[T][i] = b[i];
    rep(i, 0, m) ca[i] = subsetZeta(ca[i]), cb[i] =
    ↪ subsetZeta(cb[i]);
    rep(i, 0, n) rep(j, 0, m) rep(k, 0, m - j) {
        c[j + k][i] += ll(ca[j][i]) * cb[k][i] % mod;
        if (c[j + k][i] >= mod) c[j + k][i] -= mod;
    }
    rep(i, 0, m) c[i] = subsetZeta(c[i],true);
    vi r(n);
    rep(i, 0, n) r[i] = c[T][i];
    return r;
} // 66fa32
#undef T
```

Grafos

Caminho Euleriano. `eulerWalk(g,m,s)`: `{vértices,ids}`, ou ambos vazios se impossível a partir de `s`. IDs em $[0, m)$; em grafo não direcionado, use o mesmo ID nas duas direções. $O(n + m)$.

483eba - graph/eulerian-walk.cpp - 23 lines

```cpp
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s =
↪ 0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
```

```
            path.pb(v), walk.pb(e);
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.eb(u, id);
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin()
    ↪  + 1, walk.rend()) };
} // 483eba
```

Árvore de pontes. `comp[v]` = componente de v na árvore de pontes. Ponte uv sse `comp[u] != comp[v]`. $O(V + E)$.

c3d125 - graph/bridges.cpp - 31 lines

```
vi bridges(vector<vi> &g) {
    int n = sz(g), ti = 0, cc = 0;
    vi tin(n), low(n), st, comp;
    comp.assign(n, -1);
    auto make = [&](int v) {
        while (1) {
            int x = st.back(); st.pop_back();
            comp[x] = cc;
            if (x == v) break;
        }
        cc++;
    }; // ea296f
    auto dfs = [&](auto self, int v, int p) -> void {
        tin[v] = low[v] = ++ti;
        st.pb(v);
        int usedPar = 0;
        for (int to : g[v]) {
            if (to == p && !usedPar) { usedPar = 1; continue;
            ↪  }
            if (!tin[to]) {
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] > tin[v]) make(to);
            } else low[v] = min(low[v], tin[to]);
        }
    }; // 45b0f5
    rep(i,0,n) if (!tin[i]) {
        dfs(dfs, i, -1);
        make(i);
    }
    return comp;
} // c3d125
```

Árvore de bloco e corte. `bct(g)`: floresta bipartida entre vértices $[0, n)$ e blocos $[n, |t|)$. `|t[v]|>1` sse v é articulação. Vértices isolados também geram um bloco. $O(V+E)$.

7605c4 - graph/block-cut-tree.cpp - 35 lines

```
vector<vi> bct(vector<vi> &g) {
    int n = sz(g), ti = 0;
    vector<vi> t(n);
    vi num(n), low(n), seen(n);
    vector<pii> st;
    auto add = [&](int a, int b) {
        int id = sz(t), it = id + 1;
        t.pb({});
        while (1) {
            auto [x, y] = st.back(); st.pop_back();
            if (seen[x] != it) seen[x] = it, t[x].pb(id),
            ↪  t[id].pb(x);
            if (seen[y] != it) seen[y] = it, t[y].pb(id),
            ↪  t[id].pb(y);
            if (x == a && y == b) break;
        }
    }; // 1a9f9b
    auto dfs = [&](auto self, int v, int p) -> void {
        num[v] = low[v] = ++ti;
        int ch = 0;
        for (int to : g[v]) if (to != p) {
            if (!num[to]) {
                st.eb(v, to);
                ch++;
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= num[v]) add(v, to);
            } else if (num[to] < num[v]) {
                st.eb(v, to);
                low[v] = min(low[v], num[to]);
            }
        }
        if (p == -1 && !ch) t.pb({v}), t[v].pb(sz(t) - 1);
    }; // e48eef
    rep(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
    return t;
} // 7605c4
```

Numeração st. Assume que o grafo é biconexo após adicionar a aresta st. $O(V + E)$.

207b6a - graph/st-numbering.cpp - 26 lines

```
vi stNumber(vector<vi> &g, int s, int t) {
    int n = sz(g), ti = 0;
    vi in(n, -1), low(n), p(n, -1), by(n), sig(n, -1), num(n);
    auto dfs = [&](auto &&self, int v) -> void {
        in[v] = low[v] = ti, by[ti++] = v;
        auto go = [&](int u) {
            if (u == p[v]) return;
            if (in[u] < 0) p[u] = v, self(self, u), low[v] =
            ↪  min(low[v], low[u]);
            else low[v] = min(low[v], in[u]);
        }; // 72ea91
        if (v == s && in[t] < 0) p[t] = s, self(self, t); //
        ↪  virtual edge
        for (int u : g[v]) if (v != s || u != t) go(u);
    }; // 22a9e5
    dfs(dfs, s);
    list<int> l = {s, t};
    vector<list<int>::iterator> it(n);
    it[s] = l.begin(), it[t] = next(l.begin());
    rep(i,2,n) {
        int v = by[i], q = p[v], x = by[low[v]];
        if (sig[x] < 0) it[v] = l.insert(it[q], v), sig[q] =
        ↪  1;
        else it[v] = l.insert(next(it[q]), v), sig[q] = -1;
    }
    int i = 0;
    for (int v : l) num[v] = i++;
    return num;
} // 207b6a
```

Componentes fortemente conexas e grafo de condensação. `scc(g)`: `{cnt,comp}`, número de SCCs e componente de cada vértice; $O(V + E)$. `condGraph(g,cnt,comp)`: DAG sem arestas repetidas; $O(V + E \log(E + 1))$.

2c057a - graph/scc.cpp - 31 lines

```
pair<int, vi> scc(vector<vi> &g) {
    int n = sz(g);
    vector<vi> rg(n);
    rep (v, 0, n) for (int u : g[v]) rg[u].pb(v);
    vi vis(n), ord, comp(n, -1);
    auto dfs1 = [&](auto self, int v) -> void {
        vis[v] = 1;
        for (int u : g[v]) if (!vis[u]) self(self, u);
        ord.pb(v);
    }; // 0fb686
    auto dfs2 = [&](auto self, int v, int c) -> void {
        comp[v] = c;
        for (int u : rg[v]) if (comp[u] < 0) self(self, u, c);
    }; // 1482a1
    rep (i, 0, n) if (!vis[i]) dfs1(dfs1, i);
    reverse(all(ord));
    int cnt = 0;
    for (int v : ord) if (comp[v] < 0) dfs2(dfs2, v, cnt++);
    return {cnt, comp};
```

```
} // a8c254

vector<vi> condGraph(vector<vi> &g, int cnt, vi &comp) {
    vector<vi> dag(cnt);
    rep (v, 0, sz(g)) for (int u : g[v])
        if (comp[v] != comp[u]) dag[comp[v]].pb(comp[u]);
    rep (i, 0, cnt) {
        sort(all(dag[i]));
        dag[i].erase(unique(all(dag[i])), end(dag[i]));
    }
    return dag;
} // a5cf62
```

2-SAT. `TwoSat ts(n);` `clause(a,va,b,vb)` adiciona $(x_a = \texttt{va}) \vee (x_b = \texttt{vb})$, índices 0-based. `solve()` retorna viabilidade; só então leia `val`. $O(V + E)$.

aaa6ea - graph/2-sat.cpp - 20 lines

```
struct TwoSat {
    int n;
    vector<vi> g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    void clause(int a, bool va, int b, bool vb) {
        int x = 2*a+va, y = 2*b+vb;
        g[x ^ 1].pb(y);
        g[y ^ 1].pb(x);
    } // 2f4cde
    bool solve() {
        auto [cnt, comp] = scc(g);
        val.assign(n, 0);
        rep (i, 0, n) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
            val[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return 1;
    } // a54ac0
};
```

Dominator tree. `domTree(g,s)` retorna o dominador imediato; `-1` para inalcançáveis. Quase linear.

72f9c4 - graph/dominator-tree.cpp - 34 lines

```
vi domTree(vector<vi> &g, int s) {
    int n = sz(g), t = 0;
    vector<vi> rg(n + 1), bucket(n + 1);
    vi arr(n), rev(n + 1), par(n + 1), sdom(n + 1), dom(n +
    ↪ 1), dsu(n + 1), mn(n + 1);
    auto dfs = [&](auto &&self, int u) -> void {
        arr[u] = ++t; rev[t] = u;
        sdom[t] = dom[t] = dsu[t] = mn[t] = t;
        for (int v : g[u]) {
            if (!arr[v]) self(self, v), par[arr[v]] = arr[u];
            rg[arr[v]].pb(arr[u]);
        }
    }; // 73ed34
    auto find = [&](auto &&self, int u, int x = 0) -> int {
        if (u == dsu[u]) return x ? -1 : u;
        int v = self(self, dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[mn[dsu[u]]] < sdom[mn[u]]) mn[u] =
        ↪ mn[dsu[u]];
        return dsu[u] = v, x ? v : mn[u];
    }; // 6abce4
    dfs(dfs, s);
    for (int i = t; i; i--) {
        for (int j : rg[i]) sdom[i] = min(sdom[i],
        ↪ sdom[find(find, j)]);
        if (i > 1) bucket[sdom[i]].pb(i);
        for (int j : bucket[i]) {
            int y = find(find, j);
            dom[j] = sdom[y] == sdom[j] ? sdom[j] : y;
        }
        if (i > 1) dsu[i] = par[i];
    }
    rep(i, 2, t + 1) if (dom[i] != sdom[i]) dom[i] =
    ↪ dom[dom[i]];
    vi idom(n, -1);
    rep(i, 1, t + 1) idom[rev[i]] = rev[dom[i]];
    return idom;
} // 72f9c4
```

Árvore geradora mínima direcionada. `dmst(n,r,e)` retorna o custo da arborescência mínima, ou `-1`; $O(E \log(V))$.

849978 - graph/directed-mst.cpp - 65 lines

```
struct E { int a, b; ll w; };
struct H { int a; ll w, d = 0; H *l = 0, *r = 0; };
void P(H *x) {
    if (!x || !x->d) return;
    x->w += x->d;
    if (x->l) x->l->d += x->d;
    if (x->r) x->r->d += x->d;
    x->d = 0;
} // 089de9
H* M(H *a, H *b) {
    if (!a || !b) return a ? a : b;
    P(a), P(b);
    if (a->w > b->w) swap(a, b);
    a->r = M(a->r, b);
    swap(a->l, a->r);
    return a;
} // 995744
H* pop(H *x) {
    P(x);
    return M(x->l, x->r);
} // 9a6e69

ll dmst(int n, int r, vector<E> es) {
    vector<H*> h(n);
    for (auto e : es) h[e.b] = M(h[e.b], new H{e.a, e.w});
    vi uf(n), se(n, -1);
    iota(all(uf), 0);
    auto f = [&](auto &&f, int x) -> int {
        return uf[x] == x ? x : uf[x] = f(f, uf[x]);
    }; // 570c1e
    se[r] = r;
    vector<pair<int, ll>> st;
    ll ans = 0;
    rep(s,0,n) {
        int u = f(f, s);
        if (se[u] >= 0) continue;
        st.clear();
        while (se[u] < 0) {
            se[u] = s;
            while (h[u] && f(f, h[u]->a) == u) h[u] =
            ↪ pop(h[u]);
            if (!h[u]) return LLONG_MAX;
            P(h[u]);
            ll c = h[u]->w;
            ans += c;
            st.eb(u, c);
            u = f(f, h[u]->a);
            if (se[u] == s) {
                H *x = 0;
                int v = u;
                for (;;) {
                    auto [w, c] = st.back();
                    st.pop_back();
                    h[w]->d -= c;
                    x = M(x, h[w]);
                    uf[w] = v;
                    if (w == v) break;
                }
                u = v;
                h[u] = x;
                se[u] = -1;
            }
        }
    }
    return ans;
} // bad540
```

Matching bipartido máximo. `hopKarp(g,l,r)`: g da esquerda para a direita; inicialize `l,r` com -1. Escreve parceiros (ou -1) e retorna o tamanho do matching. $O(E\sqrt{V})$.

2d4485 - graph/hopcroft-karp.cpp - 31 lines

```cpp
int hopKarp(vector<vi> &g, vi &l, vi &r) {
    int n = sz(l), m = sz(r);
    vi d(n);
    auto bfs = [&]() {
        queue<int> q;
        rep(i, 0, n) d[i] = l[i] < 0 ? q.push(i), 0 : -1;
        bool f = 0;
        while (sz(q)) {
            int v = q.front(); q.pop();
            for (int u : g[v]) {
                int w = r[u];
                if (w < 0) f = 1;
                else if (d[w] < 0) d[w] = d[v] + 1, q.push(w);
            }
        }
        return f;
    }; // 0d5d50
    auto dfs = [&](auto &&self, int v) -> bool {
        for (int u : g[v]) {
            int w = r[u];
            if (w < 0 || (d[w] == d[v] + 1 && self(self, w)))
            ↪ {
                l[v] = u, r[u] = v;
                return 1;
            }
        }
        return d[v] = -1, 0;
    }; // a3185e
    int ans = 0;
    while (bfs()) rep(i, 0, n) ans += l[i] < 0 && dfs(dfs, i);
    return ans;
} // 2d4485
```

Matching bipartido com menor custo. `hungarian(a)`: `{custo,match}`, uma coluna distinta por linha; matriz $n \times m$, $n \le m$. $O(n^2 m)$.

c8c7c0 - graph/hungarian.cpp - 33 lines

```cpp
pair<ll, vi> hungarian(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]);
    const ll INF = 4e18;
    vector<ll> u(n + 1), v(m + 1), minv(m + 1);
    vi p(m + 1), way(m + 1), ans(n);
    rep(i, 1, n + 1) {
        p[0] = i;
        int j0 = 0;
        fill(all(minv), INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    rep(j, 1, m + 1) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans};
} // c8c7c0
```

Encontrar matching geral. `matching(g)`: parceiros em grafo não direcionado, -1 se livre. $O(n^3)$.

35862b - graph/matching.cpp - 51 lines

```cpp
vi matching(vector<vi> &g) {
    int n = sz(g), T = 0;
    vi m(n, -1), p(n), b(n), q(n), u(n), w(n), l(n);
    auto F = [&](int x, int y) {
        for (++T;; x = p[m[x]]) {
            x = b[x];
            l[x] = T;
            if (m[x] < 0) break;
        }
        for (;; y = p[m[y]]) {
            y = b[y];
            if (l[y] == T) return y;
        }
    }; // 43ebed
    auto G = [&](int x, int c, int y) {
        for (; b[x] != c; x = p[m[x]])
            w[b[x]] = w[b[m[x]]] = 1, p[x] = y, y = m[x];
    }; // 1f6aa5
    rep(s, 0, n) if (m[s] < 0) {
        fill(all(u), 0), fill(all(p), -1), iota(all(b), 0);
        int a = 0, z = 0, x = -1;
        q[z++] = s, u[s] = 1;
        while (a < z && x < 0) {
            int v = q[a++];
            for (int y : g[v]) {
                if (b[v] == b[y] || m[v] == y)
                    continue;
                if (y == s || (~m[y] && ~p[m[y]])) {
                    int c = F(v, y);
                    fill(all(w), 0);
                    G(v, c, y);
                    G(y, c, v);
                    rep(i, 0, n) if (w[b[i]])
                        b[i] = c, !u[i] && (u[i] = 1, q[z++] =
                        ↪ i);
                } else if (!~p[y]) {
                    p[y] = v;
                    if (!~m[y]) {
                        x = y;
                        break;
                    }
                    y = m[y], u[y] = 1, q[z++] = y;
                }
            }
        }
        while (~x) {
            int y = p[x], v = m[y];
            m[x] = y, m[y] = x, x = v;
        }
    }
    return m;
} // 35862b
```

Fluxo máximo (Dinic). `addEdge(u,v,cap)`: direcionada, capacidade não negativa. `flow(s,t,mx)` retorna o fluxo adicional, até `mx`; requer $s \neq t$ e mantém o residual entre chamadas. $O(V^2 E)$ para fluxo máximo.

e5f6d7 - graph/dinic.cpp - 53 lines

```cpp
struct Dinic {
    struct Edge {
        ll s, t, f, c;
    };
    vector<Edge> e;
    vector<vi> adj;
    vi ne, lvl;
    Dinic(int n): adj(n), ne(n), lvl(n) {}
    void addEdge(int a, int b, ll c) {
        adj[a].pb(sz(e));
        e.eb(a, b, 0, c);
        adj[b].pb(sz(e));
        e.eb(b, a, 0, 0);
    } // 5cff3f
    bool bfs(int s, int t) {
        vi q{s};
        fill(all(lvl), -1);
        lvl[s] = 0;
        for (int j = 0; j < sz(q); ++j) {
            int x = q[j];
            for (int i: adj[x]) {
                if (e[i].f == e[i].c) continue;
                if (lvl[e[i].t] != -1) continue;
```

```
                lvl[e[i].t] = lvl[x] + 1;
                q.pb(e[i].t);
            }
        }
        return lvl[t] == -1;
    } // f66943
    ll dfs(int x, int t, ll f) {
        if (f == 0) return 0;
        if (x == t) return f;
        for (int &te = ne[x]; te < sz(adj[x]); te++) {
            int i = adj[x][te], y = e[i].t;
            if (lvl[y] != lvl[x] + 1) continue;
            if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
                e[i].f += df;
                e[i ^ 1].f -= df;
                return df;
            }
        }
        return 0;
    } // d7b05b
    ll flow(int s, int t, ll mx = LLONG_MAX) {
        ll mf = 0;
        while (mf < mx) {
            if (bfs(s, t)) break;
            fill(all(ne), 0);
            while (ll f = dfs(s, t, mx - mf)) mf += f;
        }
        return mf;
    } // b19820
};
```

Árvore de Gomory–Hu. `gomoryHu(n,es)`: grafo não direcionado, arestas `(u,v,c)`, $c \geq 0$. Retorna árvore onde o mínimo peso no caminho u–v é seu corte mínimo. $n-1$ fluxos; desconexos ligados por peso zero.

088e89 - graph/gomory-hu.cpp - 16 lines

```
vector<tuple<int,int,ll>> gomoryHu(int n,
↪  vector<tuple<int,int,ll>> es) {
    vi par(n);
    vector<tuple<int,int,ll>> tree;
    for (int i = 1; i < n; ++i) {
        Dinic g(n);
        for (auto [u,v,c] : es) if (u != v) {
            g.addEdge(u,v,c), g.addEdge(v,u,c);
        }
        ll cut = g.flow(i,par[i]);
        g.bfs(i,par[i]);
        tree.emplace_back(i,par[i],cut);
        for (int j = i + 1; j < n; ++j)
            if (par[j] == par[i] && g.lvl[j] >= 0) par[j] = i;
    }
    return tree;
} // 088e89
```

Fluxo de custo mínimo. `addEdge(u,v,cap,cost)`; `flow(s,t,k)` retorna `{fluxo,custo}` adicional até k, mantendo o residual. Requer $s \neq t$, sem self-loops e sem ciclos negativos no residual inicial; custos negativos são permitidos. $O(VE + FE \log V)$, F = número de aumentos.

e07252 - graph/mcf.cpp - 58 lines

```
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n;
    vector<vector<E>> g;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].eb(v, sz(g[v]), cap, cost);
        g[v].eb(u, sz(g[u]) - 1, 0, -cost);
    } // 82791b
    pll flow(int s, int t, ll k = 4e18) {
        const ll INF = 4e18;
        ll fl = 0, cost = 0;
        vector<ll> p(n, INF), d(n), f(n);
        vi pv(n), pe(n), inq(n);
        queue<int> q;
        p[s] = 0, q.push(s), inq[s] = 1;
        while (sz(q)) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            rep(i,0,sz(g[u])) {
                auto &e = g[u][i];
                if (e.cap && p[e.to] > p[u] + e.cost) {
                    p[e.to] = p[u] + e.cost;
                    if (!inq[e.to]) q.push(e.to), inq[e.to] =
                    ↪  1;
                }
            }
        }
        rep(i,0,n) if (p[i] == INF) p[i] = 0;
        while (fl < k) {
            fill(all(d), INF);
            priority_queue<pll, vector<pll>, greater<pll>> pq;
            d[s] = 0, f[s] = k - fl, pq.emplace(0, s);
            while (sz(pq)) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                rep(i,0,sz(g[u])) {
                    auto &e = g[u][i];
                    if (!e.cap) continue;
                    ll nd = du + e.cost + p[u] - p[e.to];
                    if (d[e.to] > nd) {
                        d[e.to] = nd, pv[e.to] = u, pe[e.to] =
                        ↪  i;
                        f[e.to] = min(f[u], e.cap);
                        pq.emplace(nd, e.to);
                    }
                }
            }
            if (d[t] == INF) break;
            rep(i,0,n) if (d[i] < INF) p[i] += d[i];
            ll x = f[t];
            fl += x, cost += x * p[t];
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= x, g[v][e.rev].cap += x;
            }
        }
        return {fl, cost};
    } // e0d612
};
```

Árvores

Matching em árvores. Processe os vértices de baixo para cima; pareie cada vértice não raiz com seu pai se ambos estiverem livres. Produz um matching máximo em $O(n)$.

LCA. `LCA lca(g,root)` e `lca.lca(a,b)`; construção $O(n \log n)$, consulta $O(1)$.

1d44d9 - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a,
↪  b); }>;

struct LCA {
    int t = 0;
    vi tm, path, ret;
    RMQ rmq;
    LCA(vector<vi> &g, int r = 0) : tm(g.size()) {
        dfs(g, r, -1);
        rmq.build(ret);
    }
    void dfs(vector<vi> &g, int x, int pre) {
        tm[x] = t++;
        for (int y: g[x]) if (y != pre) {
            path.pb(x), ret.pb(tm[x]);
            dfs(g, y, x);
        }
    } // 2e7ca5
    int lca(int x, int y) {
        if (x == y) return x;
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

HLD. `HLD(g,r)`: árvore não vazia. Subárvore em $[\texttt{tin}[u], \texttt{tout}[u])$; `path(u,v)` retorna pares de extremos **inclusivos**, na ordem $u \to v$ (podem ser decrescentes). Construção $O(n)$; $O(\log n)$ intervalos por caminho.

c869f5 - tree/hld.cpp - 39 lines

```cpp
struct HLD {
    vi cnt, nxt, par, tin, tout, d;
    int T = 0;
    HLD(vector<vi> g, int r = 0) :
        cnt(sz(g)), nxt(sz(g)), par(sz(g), -1),
        tin(sz(g)), tout(sz(g)), d(sz(g)) {
        auto dfs1 = [&](auto &&self, int u) -> void {
            cnt[u] = 1, nxt[u] = u;
            if (par[u] != -1) g[u].erase(find(all(g[u]),
            ↪ par[u]));
            for (int &v: g[u]) {
                par[v] = u, d[v] = d[u] + 1;
                self(self, v);
                cnt[u] += cnt[v];
                if (cnt[v] > cnt[g[u][0]]) swap(v, g[u][0]);
            }
        }; // c30f49
        auto dfs2 = [&](auto &&self, int u) -> void {
            tin[u] = T++;
            for (int v: g[u]) {
                if (v == g[u][0]) nxt[v] = nxt[u];
                self(self, v);
            }
            tout[u] = T;
        }; // f292ec
        dfs1(dfs1, r), dfs2(dfs2, r);
    }
    vector<pii> path(int u, int v) {
        vector<pii> a, b;
        while (nxt[u] != nxt[v]) {
            if (d[nxt[u]] < d[nxt[v]])
                b.eb(tin[nxt[v]], tin[v]), v = par[nxt[v]];
            else
                a.eb(tin[u], tin[nxt[u]]), u = par[nxt[u]];
        }
        (d[u] < d[v] ? b : a).eb(tin[u], tin[v]);
        a.insert(end(a), rbegin(b), rend(b));
        return a;
    } // 32f9ea
};
```

Árvore virtual. `virtree(...,v)`: arestas `{pai,filho}`. `v` vira o fechamento por LCA, ordenado por `tin` e sem duplicatas; `tin,tout` da mesma raiz, subárvore $[\texttt{tin}, \texttt{tout})$. Para k vértices e LCA $O(1)$: $O(k \log k)$.

57ab2b - tree/virtual-tree.cpp - 17 lines

```cpp
vector<pii> virtree(vector<vi> &t, LCA &lca, vi &tin, vi
↪ &tout, vi &v) {
    if (v.empty()) return {};
    auto cmp = [&](int a, int b) { return tin[a] < tin[b]; };
    sort(all(v), cmp);
    int m = sz(v);
    rep(i,0,m-1) v.pb(lca.lca(v[i], v[i+1]));
    sort(all(v), cmp);
    v.erase(unique(all(v)), v.end());
    vector<pii> e;
    vi st = {v[0]};
    rep(i,1,sz(v)) {
        while (tout[st.back()] <= tin[v[i]]) st.pop_back();
        e.eb(st.back(), v[i]);
        st.pb(v[i]);
    }
    return e;
} // 57ab2b
```

Árvore de reconstrução de Kruskal. `KruskalTree(n,e)`, arestas `(peso,u,v)` não orientadas. Folhas $[0, n)$; campos: `ch` filhos, `w` peso, `cnt` número de folhas, `p` pai; raízes em `roots`, pai -1. Para folhas distintas conexas, peso do LCA = menor limiar que as conecta. Floresta desconexa permitida. $O(n + m \log m)$, até $2n - 1$ nós.

590540 - tree/kruskal-tree.cpp - 23 lines

```cpp
struct KruskalTree {
    vi p, cnt, roots;
    vector<array<int,2>> ch;
    vector<ll> w;
    KruskalTree(int n, vector<tuple<ll,int,int>> e)
        : p(n,-1), cnt(n,1), ch(n,{-1,-1}), w(n) {
        vi d(n); iota(all(d),0);
        auto f = [&](int u) {
            while (u != d[u]) u = d[u] = d[d[u]];
            return u;
        }; // 76ce70
        sort(all(e));
        for (auto [c,u,v] : e) {
            u = f(u); v = f(v); if (u == v) continue;
            int k = sz(p);
            p[u] = p[v] = k; p.pb(-1);
            ch.pb({u,v}); w.pb(c);
            cnt.pb(cnt[u]+cnt[v]);
            d[u] = d[v] = k; d.pb(k);
        }
        rep(i,0,sz(p)) if (p[i] < 0) roots.pb(i);
    }
};
```

Link-cut tree. `LinkCut<S,op,e> t(n)`: vértices **1 a n**, sem arestas; monoide como na segtree. `set(u,x)`; `query(u,v)` agrega $u \to v$, extremos inclusos, requer conexão; aceita `op` não comutativa. `link/cut(u,v)`: falso se criaria ciclo/aresta ausente. $O(\log n)$ amortizado com `op` e cópias $O(1)$; memória $O(n)$.

cf9cc7 - tree/link-cut.cpp - 70 lines

```cpp
template<class S, S (*op)(S, S), S (*e)()>
struct LinkCut {
    struct Node { int c[2] = {}, p = 0; bool rev = 0;
        S val = e(), fwd = e(), bwd = e(); };
    vector<Node> t;
    LinkCut(int n) : t(n+1) {}
    bool auxRoot(int x) {
        int p = t[x].p;
        return !p || (t[p].c[0] != x && t[p].c[1] != x);
    } // a99fc7
    void pull(int x) {
        auto &a = t[t[x].c[0]], &b = t[t[x].c[1]];
        t[x].fwd = op(op(a.fwd,t[x].val),b.fwd);
        t[x].bwd = op(op(b.bwd,t[x].val),a.bwd);
    } // 3d98f0
    void flip(int x) {
        if (x) {
            swap(t[x].c[0],t[x].c[1]);
            swap(t[x].fwd,t[x].bwd); t[x].rev ^= 1;
        }
    } // 6aa904
    void push(int x) {
        if (t[x].rev) flip(t[x].c[0]), flip(t[x].c[1]),
        ↪ t[x].rev = 0;
    } // a7444f
    void rotate(int x) {
        int p = t[x].p, g = t[p].p, d = t[p].c[1] == x;
        int b = t[x].c[d^1];
        if (!auxRoot(p)) t[g].c[t[g].c[1] == p] = x;
        t[x].p = g; t[x].c[d^1] = p; t[p].p = x;
        t[p].c[d] = b; if (b) t[b].p = p;
        pull(p); pull(x);
    } // b293c3
    void splay(int x) {
        vector<int> path{x};
        for (int y = x; !auxRoot(y); ) path.push_back(y =
        ↪ t[y].p);
        for (auto it = path.rbegin(); it != path.rend(); ++it)
        ↪ push(*it);
        while (!auxRoot(x)) {
            int p = t[x].p, g = t[p].p;
            if (!auxRoot(p)) rotate((t[p].c[1] == x) ==
                (t[g].c[1] == p) ? p : x);
            rotate(x);
```

```
        }
    } // bf3fc3
    void access(int x) {
        for (int y = x, last = 0; y; last = y, y = t[y].p) {
            splay(y); t[y].c[1] = last; pull(y);
        }
        splay(x);
    } // 5a956c
    void makeRoot(int x) { access(x); flip(x); }
    int root(int x) {
        access(x); push(x);
        while (t[x].c[0]) x = t[x].c[0], push(x);
        splay(x); return x;
    } // 398918
    bool connected(int u, int v) { return root(u) == root(v);
    ↪ }
    bool link(int u, int v) {
        makeRoot(u); if (root(v) == u) return false;
        t[u].p = v; return true;
    } // 67d999
    bool cut(int u, int v) {
        makeRoot(u); access(v);
        if (t[v].c[0] != u || t[u].c[1]) return false;
        t[v].c[0] = t[u].p = 0; pull(v); return true;
    } // 8455bd
    void set(int u, S val) { access(u); t[u].val = val;
    ↪ pull(u); }
    S query(int u, int v) {
        makeRoot(u); access(v); return t[v].fwd;
    } // ae1361
};
```

Strings

Hashing. `H()` é neutro; `H(x)` tem um símbolo. `a.concat(b)` retorna ab. Módulo $2^{61}-1$, base aleatória compartilhada; $O(1)$ por concatenação.

ddb4d3 - strings/hashing.cpp - 13 lines

```
struct H {
    static constexpr ll M = (1LL<<61)-1;
    inline static const ll base =
    ↪ 256+mt19937_64(random_device{}())()%(M-257);
    ll h = 0, pw = 1;
    int cnt = 0;
    H() = default;
    H(ll x, int n=1, ll p=base): h((x%M+M)%M), pw(p), cnt(n)
    ↪ {}
    static ll mul(ll a, ll b) { return (__int128)a*b%M; }
    H concat(H b) const {
        return {mul(h,b.pw)+b.h, cnt+b.cnt, mul(pw,b.pw)};
    } // 14f320
    bool operator==(const H&) const = default;
};
```

KMP. `kmp(s)` retorna a prefix function em $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Z-function. `zFunc(s)`: `z[i]` = LCP de `s` com `s[i..n)`; `z[0]=n`. Aceita string vazia. $O(n)$.

fc69f3 - strings/z-function.cpp - 12 lines

```
vi zFunc(string s) {
    int n = sz(s), l = 0, r = 0;
    vi z(n);
    if (!n) return z;
    rep(i, 1, n) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    z[0] = n;
    return z;
} // fc69f3
```

Aho-Corasick. `AC`: adicione padrões `a-z` com `add`; chame `build` uma vez, depois `next(v,c)`. `add` retorna nó terminal; ocorrências exigem propagação pelos links. Construção $O(26|\texttt{AC}|)$; percurso $O(|\texttt{texto}|)$.

e20924 - strings/aho-corasick.cpp - 36 lines

```
struct AC {
    static constexpr int A = 26;
    struct N {
        array<int, A> to{};
        int link = 0;
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    vi ord;
    int add(string& s) {
        int v = 0;
        for (char c : s) {
            int x = c - 'a';
            if (t[v].to[x] == -1) t[v].to[x] = sz(t), t.eb();
            v = t[v].to[x];
        }
        return v;
    } // bd83ba
    void build() {
        ord.clear();
        rep(c,0,A) {
            int u = t[0].to[c];
            if (u == -1) t[0].to[c] = 0;
            else ord.pb(u);
        }
        for (int i = 0; i < sz(ord); ++i) {
            int v = ord[i];
            rep(c,0,A) {
                int& u = t[v].to[c];
                if (u == -1) u = t[t[v].link].to[c];
                else t[u].link = t[t[v].link].to[c],
                ↪ ord.pb(u);
            }
        }
    } // 341c62
    int next(int v, char c) { return t[v].to[c - 'a']; }
};
```

Array de sufixos e LCP. `sufArr(s)` retorna `{sa,lcp}`: sem sufixo vazio; `lcp[i]` entre `sa[i-1]` e `sa[i]`, `lcp[0]=0`. Requer $n > 0$; $O(n \log n)$.

4e3b00 - strings/suffix-array.cpp - 35 lines

```
pair<vi,vi> sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s), k = 0;
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        }; // 3af78c
        srt(d); srt(c);
        e[sa[0]] = 1;
        rep(i,1,n) {
            int a = sa[i-1], b = sa[i];
            e[b] = e[a] + (c[a] != c[b] || d[a] != d[b]);
        }
```

```
        swap(c, e);
    }
    d[0] = 0;
    rep(i,0,n) {
        if (c[i] == 1) { k = 0; continue; }
        int j = sa[c[i] - 2];
        while (i + k < n && j + k < n && s[i + k] == s[j + k])
        ↪  k++;
        d[c[i] - 1] = k;
        if (k) k--;
    }
    return {sa, d};
} // 4e3b00
```

Árvore de sufixos. `ST(s,sa,lcp)`: $O(n)$.

31e155 - strings/suffix-tree.cpp - 31 lines

```
struct ST {
    struct N {
        int l, r, p, suf;
        vi ch;
    };
    vector<N> t;
    vi dep;
    int nn(int l, int r, int p, int suf = -1) {
        t.pb({l, r, p, suf, {}});
        dep.pb(p == -1 ? 0 : dep[p] + r - l);
        if (p != -1) t[p].ch.pb(sz(t) - 1);
        return sz(t) - 1;
    } // aa5bfe
    ST(string& s, vi& sa, vi& lcp) {
        int n = sz(s), rt = nn(0, 0, -1), last = rt;
        rep(i,0,n) {
            int x = sa[i], k = i ? lcp[i] : 0;
            while (dep[last] > k) last = t[last].p;
            if (dep[last] < k) {
                int y = t[last].ch.back();
                ↪  t[last].ch.pop_back();
                int z = nn(t[y].l, t[y].l + k - dep[last],
                ↪  last);
                t[y].p = z;
                t[y].l += k - dep[last];
                t[z].ch.pb(y);
                dep[z] = k;
                last = z;
            }
            last = nn(x + k, n, last, x);
        }
    }
};
```

Autômato de sufixos. `SAM sam`; `extend(c)` para letras `a-z`. `ord()` ordena estados por `len` crescente; propague `cnt` pelos links em ordem inversa. `t[v].end`: fim 0-based de uma ocorrência; comprimento L dá intervalo $[\texttt{end}-L+1, \texttt{end}+1)$. Construção $O(n)$.

73ae80 - strings/suffix-automaton.cpp - 40 lines

```
struct SAM {
    struct N {
        int len = 0, link = -1, end = -1;
        array<int, 26> to;
        ll cnt = 0; // propagate to links in reverse ord()
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    int last = 0;
    void extend(char c) {
        int x = c - 'a', cur = sz(t), p = last;
        t.eb();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1; t[cur].end = t[cur].len - 1;
        for (; p != -1 && t[p].to[x] == -1; p = t[p].link)
        ↪  t[p].to[x] = cur;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].to[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int cl = sz(t);
                t.pb(t[q]);
                t[cl].len = t[p].len + 1;
                t[cl].cnt = 0;
                for (; p != -1 && t[p].to[x] == q; p =
                ↪  t[p].link) t[p].to[x] = cl;
                t[q].link = t[cur].link = cl;
            }
        }
        last = cur;
    } // c46433
    vi ord() {
        int mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vi cnt(mx + 1), ord(sz(t));
        for (auto &u : t) cnt[u.len]++;
        rep(i,1,mx+1) cnt[i] += cnt[i - 1];
        for (int i = sz(t) - 1; i >= 0; i--)
        ↪  ord[--cnt[t[i].len]] = i;
        return ord;
    } // a8d426
};
```

Manacher. `manacher(s)`: `{d1,d2}`. Comprimentos $2\texttt{d1}[i]-1$ (centro i) e $2\texttt{d2}[i]$ (centro entre $i-1, i$). $O(n)$.

59be6a - strings/manacher.cpp - 17 lines

```
pair<vi, vi> manacher(string& s) {
    int n = sz(s);
    vi d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i +
        ↪  k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] ==
        ↪  s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2}; // {odd, even}
} // 59be6a
```

Decomposição de Lyndon. `lyndon(s)`: fronteiras $p = (0, \ldots, n)$ da fatoração; fatores $s[p_i, p_{i+1})$ em ordem lexicográfica não crescente. String vazia retorna `{0}`. $O(n)$.

751105 - strings/lyndon.cpp - 14 lines

```
vi lyndon(string s) {
    int n = sz(s);
    vi p = {0};
    for (int i = 0; i < n;) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) p.pb(i += j - k);
    }
    return p;
} // 751105
```

Encontrar runs em uma string. `getRuns(s)`: triplas `{p,l,r}` de repetições maximais em $[l, r)$, período mínimo p, $r - l \geq 2p$. $O(n \log^2 n)$ nesta implementação.

934209 - strings/runs.cpp - 38 lines

```
using tiii = tuple<int, int, int>;

vector<tiii> getRuns(string &S) {
    int N = sz(S);
    vector<vector<pii>> B(N + 1);
    auto go = [&](string &l, string &r, int M, int o) {
        int n = sz(l), m = sz(r);
```

```
        string s = l, t = r + l + r;
        reverse(all(s));
        auto z = zFunc(s), w = zFunc(t);
        rep(p,1,n+1) {
            int a = p == n ? n : min(p + z[p], n), b = min(w[n
            ↪  + m - p], m);
            if (a + b >= 2 * p) B[p].eb(M - (o ? b : a), M +
            ↪  (o ? a : b));
        }
    }; // 9dd4a1
    auto dfs = [&](auto &&self, int L, int R) -> void {
        if (R - L < 2) return;
        int M = (L + R) >> 1;
        self(self, L, M), self(self, M, R);
        string l(begin(S) + L, begin(S) + M), r(begin(S) + M,
        ↪  begin(S) + R);
        go(l, r, M, 0);
        reverse(all(l)), reverse(all(r));
        go(r, l, M, 1);
    }; // dea6cf
    dfs(dfs, 0, N);
    vector<tiii> res;
    set<pii> done;
    rep(p,0,N+1) {
        auto &v = B[p];
        sort(all(v), [](pii a, pii b) { return a.fi == b.fi ?
        ↪  a.se > b.se : a.fi < b.fi; });
        int R = -1;
        for (auto [l, r] : v) if (R < r) {
            R = r;
            if (done.emplace(l, r).se) res.eb(p, l, r);
        }
    }
    return res;
} // d4dc26
```

Geometria

Base. `Pt=complex<ll>`; produtos devem caber em `ll`. Inteiros: `EPS=0`. Nas entradas marcadas *Reais*, troque `Pt`, `dot` e `cross` para `ld`.

673f55 - geometry/core.cpp - 9 lines

```
using Pt = complex<ll>;
#define xx real()
#define yy imag()

ll dot(Pt a, Pt b) { return (conj(a) * b).xx; }
ll cross(Pt a, Pt b) { return (conj(a) * b).yy; }
Pt perp(Pt a) { return Pt(-a.yy, a.xx); }
const ld EPS = 1e-9;
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
```

Interseção de retas. `lineInter`: Reais; retas infinitas ab, cd, não paralelas e não degeneradas.

2fc473 - geometry/line-intersection.cpp - 3 lines

```
Pt lineInter(Pt a, Pt b, Pt c, Pt d) {
    return a + (b - a) * cross(c - a, d - c) / cross(b - a, d
    ↪  - c);
} // 2fc473
```

Interseção de segmentos. `segInter` inclui extremos; `properInter` exige cruzamento estrito.

f6de72 - geometry/segment-intersection.cpp - 16 lines

```
bool segInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    if (ab_c * ab_d < 0 && cd_a * cd_b < 0) return 1;
    if (!ab_c && sgn(dot(c - a, c - b)) <= 0) return 1;
    if (!ab_d && sgn(dot(d - a, d - b)) <= 0) return 1;
    if (!cd_a && sgn(dot(a - c, a - d)) <= 0) return 1;
    if (!cd_b && sgn(dot(b - c, b - d)) <= 0) return 1;
    return 0;
} // 4f106a

bool properInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
} // 66a13a
```

Área orientada. `polyArea`: dobro da área orientada; positivo em sentido anti-horário. $O(n)$.

b23e06 - geometry/polygon-area.cpp - 5 lines

```
ll polyArea(vector<Pt> &pt) { // ld: change ret type to ld
    ll n = sz(pt), s = 0; // ld: change to ld s
    rep (i, 0, n) s += cross(pt[i], pt[(i + 1) % n]);
    return s;
} // b23e06
```

Convex hull. `convHull`: índices do hull anti-horário; exclui colineares nas arestas. Remova duplicatas e trate $n \leq 1$ fora. $O(n \log n)$.

b43fd9 - geometry/convex-hull.cpp - 22 lines

```
vi convHull(vector<Pt> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                Pt a = pt[st[m - 1]], b = pt[st[m - 2]], c =
                ↪  pt[i];
                if (cross(b - a, c - a) < 0) break; // 1) >
                ↪  for clockwise, <= to include non-vertices
                st.pop_back();
            }
            st.pb(i);
        }
        st.pop_back();
        h.insert(h.end(), all(st));
    }; // 1938a0
    iota(all(ord), 0);
    auto top = [](auto a) { return pair(a.xx, a.yy); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) >
    ↪  top(pt[j]); });
    add(), reverse(all(ord)), add();
    return h;
} // b43fd9 // 1) ld: use sgn(cross(...)) < 0
```

Ponto em convexo. `pointInPoly`: convexo anti-horário, $n \geq 3$, sem repetir o primeiro vértice nem manter colineares consecutivos. `border=true` inclui a borda. $O(\log n)$.

1ce015 - geometry/point-in-polygon.cpp - 10 lines

```
bool pointInPoly(vector<Pt> &pt, Pt q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1; // ld: ld
    ↪  t = border ? EPS : 0
    auto v = [&](int i) { return cross(q - pt[0], pt[i] -
    ↪  pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m; // ld: use v(m) > EPS
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t; // ld: use the
    ↪  ld t above
} // 1ce015
```

Corte de polígono. `polyCut`: parte do polígono à esquerda da reta orientada $a \to b$, incluindo a borda; $a \neq b$. Reais. $O(n)$.

a60a48 - geometry/polygon-cut.cpp - 10 lines

```
vector<Pt> polyCut(vector<Pt> p, Pt a, Pt b) {
    vector<Pt> q;
    rep(i,0,sz(p)) {
        Pt c = p[i], d = p[(i + 1) % sz(p)];
```

```
        int sc = sgn(cross(b - a, c - a)), sd = sgn(cross(b -
        ↪  a, d - a));
        if (sc >= 0) q.pb(c);
        if (sc * sd < 0) q.pb(lineInter(a, b, c, d));
    }
    return q;
} // a60a48
```

Interseção de semiplanos. `Halfplane(a,b)`: esquerda de $a \to b$, $a \neq b$. `halfplaneInter` intersecta com $[-B, B]^2$, $B > 0$: polígono anti-horário, ou vazio se sem área positiva. Não distingue regiões ilimitadas. Reais; tolerância `EPS`, paralelos permitidos. $O(n \log n)$.

a56079 - geometry/halfplane-intersection.cpp - 34 lines

```
struct Halfplane {
    Pt p, d; ld angle;
    Halfplane(Pt a, Pt b): p(a), d((b-a)/abs(b-a)),
    ↪  angle(arg(d)) {}
    bool out(Pt q) const { return cross(d,q-p) < -EPS; }
};
Pt hpInter(Halfplane a, Halfplane b) {
    return a.p + a.d * (cross(b.p-a.p,b.d)/cross(a.d,b.d));
} // f8014a
vector<Pt> halfplaneInter(vector<Halfplane> h, ld bound) {
    vector<Pt> box = {{-bound,-bound},{bound,-bound},
        {bound,bound},{-bound,bound}};
    rep(i,0,4) h.emplace_back(box[i],box[(i+1)%4]);
    sort(all(h), [](auto a, auto b) { return a.angle <
    ↪  b.angle; });
    vector<Halfplane> q = h;
    int l = 0, r = 0;
    for (auto a : h) {
        while (r-l>1 && a.out(hpInter(q[r-2],q[r-1]))) --r;
        while (r-l>1 && a.out(hpInter(q[l],q[l+1]))) ++l;
        if (r>l && abs(cross(a.d,q[r-1].d)) < EPS) {
            if (dot(a.d,q[r-1].d) < 0) return {};
            if (!a.out(q[r-1].p)) continue;
            --r;
        }
        q[r++] = a;
    }
    while (r-l>2 && q[l].out(hpInter(q[r-2],q[r-1]))) --r;
    while (r-l>2 && q[r-1].out(hpInter(q[l],q[l+1]))) ++l;
    if (r-l<3) return {};
    vector<Pt> p;
    rep(i,l,r) p.pb(hpInter(q[i],q[i+1<r ? i+1 : l]));
    ld area = 0;
    rep(i,0,sz(p)) area +=
    ↪  cross(p[i]-p[0],p[(i+1)%sz(p)]-p[0]);
    return area > EPS ? p : vector<Pt>{};
} // 4682fa
```

Soma de Minkowski. `minkowski`: convexos anti-horários, sem vértice final repetido; saída anti-horária, podendo manter colineares. $O(|a| + |b|)$.

09f042 - geometry/minkowski.cpp - 17 lines

```
vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) {
    auto norm = [&](vector<Pt>& p) {
        auto lt = [](Pt a, Pt b) { return a.xx != b.xx ? a.xx
        ↪  < b.xx : a.yy < b.yy; };
        rotate(p.begin(), min_element(all(p), lt), p.end());
        p.pb(p[0]), p.pb(p[1]);
    }; // f5b3e9
    int n = sz(a), m = sz(b);
    norm(a), norm(b);
    vector<Pt> c;
    for (int i = 0, j = 0; i < n || j < m; ) {
        c.pb(a[i] + b[j]);
        ll z = cross(a[i + 1] - a[i], b[j + 1] - b[j]); // ld:
        ↪  keep z in ld
        if (z >= 0 && i < n) i++; // ld: use z >= -EPS
        if (z <= 0 && j < m) j++; // ld: use z <= EPS
    }
    return c;
} // 09f042 //ld: a.xx != b.xx -> sgn(a.xx - b.xx)
```

Círculo–reta. `circLineInter`: pontos do círculo com a reta $a + td$, $d \neq 0$. Reais.

21e90b - geometry/circle-line.cpp - 8 lines

```
vector<Pt> circLineInter(Pt c, ld r, Pt a, Pt d) {
    Pt f = a + d * dot(c - a, d) / norm(d);
    ld h2 = r * r - norm(c - f);
    if (sgn(h2) < 0) return {};
    ld dt = sqrt(max((ld)0, h2)) / abs(d);
    if (!sgn(h2)) return {f};
    return {f - dt * d, f + dt * d};
} // 21e90b
```

Círculo–círculo. `circCircInter`: Reais; trate círculos coincidentes fora.

d3b1f8 - geometry/circle-circle.cpp - 12 lines

```
vector<Pt> circCircInter(Pt c1, ld r1, Pt c2, ld r2) {
    Pt d = c2 - c1;
    ld dist = abs(d);
    if (!sgn(dist)) return {};
    ld a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    ld h2 = r1 * r1 - a * a;
    if (sgn(h2) < 0) return {};
    Pt mid = c1 + d * (a / dist);
    if (!sgn(h2)) return {mid};
    Pt p = perp(d) * (sqrt(max((ld)0, h2)) / dist);
    return {mid - p, mid + p};
} // d3b1f8
```

Tangentes comuns. `circTangents`: pares de pontos de tangência, externas e internas. Reais; trate centros iguais fora e remova duplicatas se necessário.

0d00d5 - geometry/circle-tangents.cpp - 14 lines

```
vector<pair<Pt, Pt>> circTangents(Pt c1, ld r1, Pt c2, ld r2)
↪  {
    vector<pair<Pt, Pt>> res;
    for (int s: {1, -1}) {
        ld r = s * r2, dr = r1 - r;
        Pt d = c2 - c1;
        ld l2 = norm(d), h2 = max((ld)0, l2 - dr * dr);
        if (sgn(l2 - dr * dr) < 0) continue;
        for (int t: {1, -1}) {
            Pt n = (dr * d + perp(d) * (t * sqrt(h2))) / l2;
            res.pb({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // 0d00d5
```

Círculo mínimo. `mec`: círculo `{o,r}` mínimo. Reais. Vetor vazio retorna raio -1. $O(n)$ esperado.

fd30f4 - geometry/enclosing-circle.cpp - 33 lines

```
struct C {
    Pt o;
    ld r;
};

bool in(C c, Pt p) { return abs(p - c.o) <= c.r + EPS; }

C circ(Pt a, Pt b) { return {(a + b) / (ld)2, abs(a - b) / 2};
↪  }

C circ(Pt a, Pt b, Pt c) {
    if (fabsl(cross(b - a, c - a)) < EPS) {
        C x = circ(a, b), y = circ(a, c), z = circ(b, c);
        if (in(x, c)) return x;
        if (in(y, b)) return y;
        return z;
    }
    Pt o = a + perp((b - a) * norm(c - a) - (c - a) * norm(b -
    ↪  a))
        / cross(b - a, c - a) / (ld)2;
    return {o, abs(a - o)};
} // a6aab7

C mec(vector<Pt> p) {
    shuffle(all(p), rng);
```

```
    C c{{0, 0}, -1};
    rep(i,0,sz(p)) if (!in(c, p[i])) {
        c = {p[i], 0};
        rep(j,0,i) if (!in(c, p[j])) {
            c = circ(p[i], p[j]);
            rep(k,0,j) if (!in(c, p[k])) c = circ(p[i], p[j],
            ↪  p[k]);
        }
    }
    return c;
} // effcbe
```

Par mais próximo. `closestPair`: par de pontos, não índices. Requer $n \geq 2$ e coordenadas inteiras; trate duplicatas antes. Distâncias ao quadrado devem caber em `ll`. $O(n \log n)$.

0c44eb - geometry/closest-pair.cpp - 20 lines

```
bool cmpX(Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx :
↪  a.yy < b.yy; }

pair<Pt,Pt> closestPair(vector<Pt> p) {
    sort(all(p), cmpX);
    set<pair<ll,ll>> s;
    pair<Pt,Pt> ret{p[0], p[1]};
    ll best = norm(p[0] - p[1]);
    for (int l = 0, i = 0; i < sz(p); i++) {
        ll d = sqrtl((ld)best) + 1;
        while (p[i].xx - p[l].xx > d) s.erase({p[l].yy,
        ↪  p[l].xx}), l++;
        auto it1 = s.lower_bound({p[i].yy - d, LLONG_MIN});
        auto it2 = s.upper_bound({p[i].yy + d, LLONG_MAX});
        for (auto it = it1; it != it2; ++it) {
            Pt q(it->se, it->fi);
            if (ll cur = norm(p[i] - q); cur < best) best =
            ↪  cur, ret = {p[i], q};
        }
        s.insert({p[i].yy, p[i].xx});
    }
    return ret;
} // 5f8da5
```

MST Manhattan. `manhattanMST`: até $4n$ arestas `(peso,u,v)` contendo uma MST de $|\Delta x| + |\Delta y|$. IDs originais, aceita duplicatas. `Pt=complex<ll>`, $|x|, |y| \leq 10^{18}$. $O(n \log n)$ tempo, $O(n)$ memória.

de0e83 - geometry/manhattan-mst.cpp - 22 lines

```
vector<tuple<ll,int,int>> manhattanMST(vector<Pt> p) {
    vi id(sz(p)); iota(all(id),0);
    vector<tuple<ll,int,int>> edges;
    rep(k,0,4) {
        sort(all(id), [&](int i,int j) {
            return p[i].xx+p[i].yy < p[j].xx+p[j].yy;
        } /* 29cc89 */);
        map<ll,int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-p[i].yy); it !=
            ↪  sweep.end();) {
                int j = it->second;
                Pt d = p[i]-p[j];
                if (d.yy > d.xx) break;
                edges.emplace_back(d.xx+d.yy,i,j);
                it = sweep.erase(it);
            }
            sweep[-p[i].yy] = i;
        }
        for (Pt &v : p) v = k&1 ? Pt(-v.xx,v.yy) :
        ↪  Pt(v.yy,v.xx);
    }
    return edges;
} // de0e83
```

Convex hull 3D. `hull3D(p)`: faces triangulares com IDs originais, orientadas para fora; aceita duplicatas e faces coplanares trianguladas. Inteiros $|x|, |y|, |z| \leq 10^9$. Vazio se dimensão afim < 3. Tempo esperado $O(n^2)$, memória $O(n)$. `volume6(a,b,c,d)>0`: d externo à face abc.

01226b - geometry/hull-3d.cpp - 55 lines

```
struct Point3 {
    ll x,y,z;
    Point3 operator-(Point3 b) const { return
    ↪  {x-b.x,y-b.y,z-b.z}; }
};
__int128 volume6(Point3 a, Point3 b, Point3 c, Point3 d) {
    b = b-a; c = c-a; d = d-a;
    return (__int128)b.x*(
    ↪  (__int128)c.y*d.z-(__int128)c.z*d.y)
        - (__int128)b.y*( (__int128)c.x*d.z-(__int128)c.z*d.x)
        + (__int128)b.z*(
        ↪  (__int128)c.x*d.y-(__int128)c.y*d.x);
} // 68c5ad
vector<array<int,3>> hull3D(const vector<Point3> &p) {
    int n = sz(p);
    if (n < 4) return {};
    vi id(n); iota(all(id),0);
    auto noncollinear = [&](int a,int b,int c) {
        auto u = p[b]-p[a], v = p[c]-p[a];
        return (__int128)u.x*v.y != (__int128)u.y*v.x ||
            (__int128)u.x*v.z != (__int128)u.z*v.x ||
            (__int128)u.y*v.z != (__int128)u.z*v.y;
    }; // 167577
    int b = 1;
    while (b < n && p[b].x == p[0].x && p[b].y == p[0].y &&
    ↪  p[b].z == p[0].z) ++b;
    if (b == n) return {};
    swap(id[1],id[b]); b = 2;
    while (b < n && !noncollinear(id[0],id[1],id[b])) ++b;
    if (b == n) return {};
    swap(id[2],id[b]); b = 3;
    while (b < n &&
    ↪  !volume6(p[id[0]],p[id[1]],p[id[2]],p[id[b]])) ++b;
    if (b == n) return {};
    swap(id[3],id[b]);
    vector<array<int,3>> f;
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4) {
        array<int,3> t = {id[i],id[j],id[k]};
        if (volume6(p[t[0]],p[t[1]],p[t[2]],p[id[6-i-j-k]]) >
        ↪  0) swap(t[1],t[2]);
        f.push_back(t);
    }
    rep(k,4,n) {
        unordered_set<uint64_t> boundary;
        vector<array<int,3>> keep;
        for (auto t : f) {
            if (volume6(p[t[0]],p[t[1]],p[t[2]],p[id[k]]) <=
            ↪  0) {
                keep.push_back(t); continue;
            }
            rep(j,0,3) {
                uint64_t a = t[j], b = t[(j+1)%3];
                if (!boundary.erase((b<<32)|a))
                ↪  boundary.insert((a<<32)|b);
            }
        }
        for (auto e : boundary)
            keep.push_back({int(e>>32),
                int(uint32_t(e)),id[k]});
        f.swap(keep);
    }
    return f;
} // a9abed
```

Fórmulas, Teoremas e Fatos

Teoria dos números

Máximos de divisores e fatores primos. Para $1 \leq n \leq N$, sejam $D(N) = \max d(n)$ e $W(N) = \max \omega(n)$, onde $d(n)$ conta divisores positivos e $\omega(n)$ conta primos distintos.

N	$D(N)$	$W(N)$	N	$D(N)$	$W(N)$
10^1	4	2	10^{10}	2304	10
10^2	12	3	10^{11}	4032	10
10^3	32	4	10^{12}	6720	11

10^4	64	5	10^{13}	10752	12
10^5	128	6	10^{14}	17280	12
10^6	240	7	10^{15}	26880	13
10^7	448	8	10^{16}	41472	13
10^8	768	8	10^{17}	64512	14
10^9	1344	9	10^{18}	103680	15
$2^{31}-1$	1600	9	$2^{63}-1$	161280	15
$2^{32}-1$	1920	9	$2^{64}-1$	184320	15

$W(N)$ é o maior k com $2\cdot 3\cdot 5\cdots p_k \le N$ (produto dos primeiros k primos).

Triplas pitagóricas. Soluções positivas primitivas de $a^2+b^2=c^2$:

$$a=m^2-n^2, \qquad b=2mn, \qquad c=m^2+n^2,$$

Para inteiros $m>n\ge 1$, $\gcd(m,n)=1$ e $m\not\equiv n \pmod 2$. Todas as soluções positivas vêm de múltiplos $k\ge 1$ e troca dos catetos.

Somas de dois quadrados. Para $n>0$, existem inteiros x,y com $x^2+y^2=n$ sse todo primo $p\equiv 3 \pmod 4$ tem expoente par na fatoração de n. Nesse caso, o número de pares *ordenados e com sinal*, incluindo coordenadas zero, é

$$r_2(n)=4\prod_{\substack{p\mid n\\ p\equiv 1 \pmod 4}}(v_p(n)+1),$$

Se a condição falha, $r_2(n)=0$. Para $n=0$, há apenas $(0,0)$.

Somas de três quadrados. Um inteiro não negativo n pode ser escrito como

$$n=x^2+y^2+z^2$$

com x,y,z inteiros sse

$$n\neq 4^a(8b+7)$$

para todos os inteiros $a,b\ge 0$.

Teorema de Lucas. Para primo p, se

$$n=\sum n_i p^i, \qquad k=\sum k_i p^i,$$

então

$$\binom{n}{k}\equiv\prod_i\binom{n_i}{k_i} \pmod p.$$

Teorema de Kummer. O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n-k$ na base p.

LTE (versão comum). Se p é um primo ímpar, $p\mid(a-b)$ e $p\nmid ab$, então

$$v_p(a^n-b^n)=v_p(a-b)+v_p(n).$$

Para a versão da soma, exija p primo ímpar, $p\nmid ab$, n ímpar e $p\mid(a+b)$:

$$v_p(a^n+b^n)=v_p(a+b)+v_p(n).$$

Inversão de Möbius. Se

$$g(n)=\sum_{d\mid n} f(d),$$

então

$$f(n)=\sum_{d\mid n}\mu(d)\,g\!\left(\frac{n}{d}\right).$$

Existência de raiz primitiva. Existem raízes primitivas módulo n sse

$$n\in\{1,2,4,p^k,2p^k\},$$

onde p é um primo ímpar.

Contagem

Número de Catalan.

$$C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}$$

Reflexão: caminhos acima de uma barreira. O número de caminhadas de $n\ge 0$ passos ± 1, de altura inteira $a\ge 0$ até $b\ge 0$, sem ficar negativo, é

$$\binom{n}{(n+b-a)/2}-\binom{n}{(n+b+a+2)/2}.$$

Binomial com índice inferior não inteiro ou fora de $[0,n]$ vale zero. Para barreira h, subtraia h das alturas.

Número de Narayana. A quantidade de caminhos de $(0,0)$ a (N,N) que não passam acima da diagonal e com exatamente M picos locais é

$$\frac{1}{N}\binom{N}{M}\binom{N}{M-1}$$

Lema dos ciclos. Para inteiros $A_i\le 1$ com soma $K>0$, exatamente K posições iniciais dão rotações com todos os prefixos positivos (conte posições, não sequências distintas).

Desarranjos.

$$D_n=n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}$$

e

$$D_0=1,\quad D_1=0,\quad D_n=(n-1)(D_{n-1}+D_{n-2})\ (n\ge 2)$$

Funções de parking. Para $1\le m\le n$, o número de sequências $a=(a_1,a_2,\ldots,a_m)$ tais que $\forall i, 1\le a_i\le n$, e para $k=1,2,\ldots,n$, o número de índices i que satisfazem $a_i\le k$ é pelo menos $m-n+k$ é $(n+1-m)(n+1)^{m-1}$

Números de Stirling do primeiro tipo. $\begin{bmatrix}n\\k\end{bmatrix}$ é o número de permutações de tamanho n com exatamente k ciclos.

$$\begin{bmatrix}n\\k\end{bmatrix}=\begin{bmatrix}n-1\\k-1\end{bmatrix}+(n-1)\begin{bmatrix}n-1\\k\end{bmatrix}$$

Números de Stirling do segundo tipo. $\begin{Bmatrix}n\\k\end{Bmatrix}$ é o número de formas de dividir os inteiros de 1 a n em k caixas não vazias e indistinguíveis.

$$\begin{Bmatrix}n\\k\end{Bmatrix}=\begin{Bmatrix}n-1\\k-1\end{Bmatrix}+k\begin{Bmatrix}n-1\\k\end{Bmatrix}$$

Forma fechada:

$$\begin{Bmatrix}n\\k\end{Bmatrix}=\frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

Fatoriais subindo. Para $x^{\overline{n}}=x(x+1)(x+2)\ldots(x+n-1)$,

temos que

$$x^{\overline{n}} = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

e

$$x^n = \sum_{k=0}^{n} (-1)^{n-k} \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\overline{k}}$$

Partições inteiras (Euler). Se $p(n)$ conta somas não ordenadas de inteiros positivos com total n, ponha $p(0) = 1$ e $p(n) = 0$ para $n < 0$. Para $n > 0$, com $g_k^{\pm} = k(3k \pm 1)/2$,

$$p(n) = \sum_{k \geq 1} (-1)^{k-1} \big(p(n - g_k^-) + p(n - g_k^+)\big).$$

Pare quando $g_k^- > n$. Calcula $p(0), \ldots, p(N)$ em $O(N\sqrt{N})$ tempo e $O(N)$ memória; válido módulo qualquer inteiro positivo.

Inversão de Lagrange. Extrai coeficientes de séries definidas implicitamente. Se $F(x) = x\phi(F(x))$, $F(0) = 0$, $\phi(0) \neq 0$, então, para $n \geq k \geq 1$,

$$[x^n]F(x)^k = \frac{k}{n}[t^{n-k}]\phi(t)^n.$$

Mais geralmente, $[x^n]H(F(x)) = \frac{1}{n}[t^{n-1}]H'(t)\phi(t)^n$ para $n \geq 1$. Em característica zero; módulo primo p, use $n < p$.

Contagem: simetrias e restrições

Permanente: contar matchings perfeitos (Ryser). Para A de tamanho $n \times n$, $n \geq 1$, o permanente soma $\prod_i A_{i,\pi(i)}$ sobre todas as permutações π. Para a matriz de adjacência bipartida 0/1, conta matchings perfeitos. Com $[n] = \{1, \ldots, n\}$,

$$\operatorname{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_{i=1}^{n} \left(\sum_{j \in S} A_{ij} \right).$$

Gray code mantém somas de linha e recalcula o produto em $O(n)$ por subconjunto. Tempo $O(n2^n)$, memória auxiliar $O(n)$; sem divisões, válido módulo qualquer inteiro positivo. Para $n = 0$, o permanente é 1.

Lema de Burnside. Para contar configurações distintas a menos de simetrias de um grupo finito G, tire a média do número de configurações fixadas por cada simetria, $|\operatorname{Fix}(g)|$:

$$\#\text{classes} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|.$$

Módulo m, só divida por inverso se $\gcd(|G|, m) = 1$; caso contrário, calcule a soma módulo $m|G|$ e divida o representante por $|G|$ nos inteiros.

Exatamente k propriedades. Seja S_j a soma das quantidades de objetos que satisfazem simultaneamente cada escolha de j entre m propriedades; S_0 é o total de objetos. O número que satisfaz *exatamente* k é

$$\sum_{j=k}^{m} (-1)^{j-k} \binom{j}{k} S_j.$$

Um objeto com t propriedades contribui $\binom{t}{j}$ a S_j; não conte apenas uma vez a união das interseções.

Árvores rotuladas (Prüfer). Para $n \geq 2$ vértices distintos, há n^{n-2} árvores. Com graus prescritos $d_i \geq 1$, $\sum d_i = 2n - 2$, há

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i - 1)!};$$

fora dessas condições, zero. Cada vértice aparece $d_i - 1$ vezes no código de Prüfer, de comprimento $n - 2$.

Ordens válidas em floresta enraizada. Em uma floresta fixa com n vértices distintos, o número de ordens em que cada pai aparece antes dos filhos é

$$\frac{n!}{\prod_v s_v}, \qquad s_v = \text{tamanho da subárvore incluindo } v.$$

Não identifique ordens por simetrias. Para usar inversos módulo primo p, exija $n < p$.

Completar uma floresta fixa. Uma floresta sobre os n vértices rotulados, com $c \geq 2$ componentes de tamanhos $s_1, \ldots, s_c$, está contida em

$$n^{c-2} \prod_{i=1}^{c} s_i$$

árvores no grafo completo. Para $c = 1$, há uma única conclusão: a própria árvore.

Somas e sequências

Somas de potências comuns.

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

Diferenças finitas e somas polinomiais. Para P de grau d, defina $\Delta P(x) = P(x+1) - P(x)$. Para inteiro $n \geq 0$,

$$P(n) = \sum_{j=0}^{d} \binom{n}{j} \Delta^j P(0),$$

$$\sum_{i=0}^{n-1} P(i) = \sum_{j=0}^{d} \binom{n}{j+1} \Delta^j P(0).$$

$\Delta^j P(0)$: primeiro elemento da j-ésima linha de diferenças de $P(0), \ldots, P(d)$. Com fatoriais inversos módulo primo p, requer $d + 1 < p$.

Convolução binomial (Vandermonde).

$$\sum_{k} \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}.$$

Inversão binomial.

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

Somas de Newton. Para $a_n x^n + \cdots + a_0$, $a_n \neq 0$, e $p_k = \sum_{j=1}^{n} x_j^k$ (raízes com multiplicidade):

$$a_n p_k + \sum_{j=1}^{k-1} a_{n-j} p_{k-j} + k a_{n-k} = 0 \quad (1 \leq k \leq n).$$

Para $k > n$: $\sum_{j=0}^{n} a_{n-j} p_{k-j} = 0$; $p_0 = n$.

Tabela de funções geradoras

$S(n,k)$: Stirling do segundo tipo, $k \geq 0$ fixo.

$[x^n]F(x)$	$F(x)$
1	$\dfrac{1}{1-x}$
a^n	$\dfrac{1}{1-ax}$
n	$\dfrac{x}{(1-x)^2}$
$\binom{n+k}{k}$	$\dfrac{1}{(1-x)^{k+1}}$
$\binom{n-a+k-1}{k-1}$	$\dfrac{x^a}{(1-x)^k}$
$\binom{k}{n}$	$(1+x)^k$
$S(n,k)$	$\dfrac{x^k}{\prod_{j=1}^{k}(1-jx)}$
$\dfrac{S(n,k)}{n!}$	$\dfrac{(e^x-1)^k}{k!}$
C_n	$\dfrac{1-\sqrt{1-4x}}{2x}$
F_n $(F_0 = F_1 = 1)$	$\dfrac{1}{1-x-x^2}$
$\dfrac{D_n}{n!}$	$\dfrac{e^{-x}}{1-x}$
$\dfrac{1}{n}$	$\log\dfrac{1}{1-x}$

Identidades para convolução

Convolução linear: $(A*B)_k = \sum_i A_i B_{k-i}$, com coeficientes fora do vetor iguais a zero.

Correlação (deslocamentos). Para a de tamanho $n>0$ e b de tamanho $m>0$, inverta a: $A_i = a_{n-1-i}$. Se $C = A*b$, então

$$\sum_i a_i b_{i+d} = C_{n-1+d}, \qquad -(n-1) \leq d \leq m-1.$$

Convolução binomial. Para obter $c_k = \sum_i \binom{k}{i} a_i b_{k-i}$, convolua $A_i = a_i/i!$, $B_i = b_i/i!$ e recupere $c_k = k!(A*B)_k$.

Translação de polinômio (Taylor shift). Para $P(x) = \sum_{i=0}^{n} a_i x^i$, forme $A_{n-i} = i! a_i$ e $B_j = c^j/j!$ $(0 \leq i,j \leq n)$. Se $C = A*B$, o coeficiente de x^k em $P(x+c)$ é $C_{n-k}/k!$, para $0 \leq k \leq n$.

Nas duas últimas receitas, os fatoriais usados devem ser invertíveis. Módulo primo p, exija grau de saída $< p$ na convolução binomial e $n < p$ no Taylor shift.

Atualização de matriz por posto 1

Lema do determinante e Sherman–Morrison: para A invertível, atualize determinante e inversa após $A \leftarrow A + uv^T$ em $O(n^2)$ se A^{-1} já é conhecida. Defina $z = A^{-1}u$, $w^T = v^T A^{-1}$ e $d = 1 + v^T z$:

$$\det(A+uv^T) = \det(A)\, d, \qquad (A+uv^T)^{-1} = A^{-1} - \frac{zw^T}{d}.$$

A fórmula da inversa requer $d \neq 0$. Válido sobre um corpo, inclusive módulo primo.

Combinatória e otimização

Teorema de Dilworth. Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.

Erdős–Szekeres. Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.

Teorema de Mirsky. Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.

Intervalos disjuntos e pontos de cobertura. Para intervalos fechados, o máximo de intervalos dois a dois disjuntos é o mínimo de pontos que intersectam todos. Ordene pelo extremo direito: escolha o primeiro intervalo ainda não coberto, marque seu extremo direito e descarte os que contêm esse ponto. Para selecionar intervalos semiabertos $[l,r)$ disjuntos, aceite $l \geq r_{\text{último}}$.

Maior soma com descarte. Cada escolhido deve ser pareado com um índice anterior distinto não escolhido, sem reutilização. Percorra 0-based, insira a_i em um min-heap e retire o mínimo em cada índice par. Ao final, some os valores mantidos; se escolher é opcional, some só os positivos.

DP

DP contando subsequências. $\texttt{DP}[i]$ conta subsequências distintas do prefixo de tamanho i, incluindo a vazia; $\texttt{DP}[0] = 1$. Se j é a última ocorrência anterior de s_i (índices a partir de 1),

$$\texttt{DP}[i] = 2\texttt{DP}[i-1] - \begin{cases} \texttt{DP}[j-1], & j \text{ existe}, \\ 0, & \text{caso contrário.} \end{cases}$$

Convolução min-plus. Para sequências convexas, mescle as diferenças em ordem crescente; o resultado também é convexo.

Grafos e jogos

Havel–Hakimi. Para realizar uma sequência de graus de grafo simples, retire um vértice de maior grau d, ligue-o aos d vértices restantes de maior grau e decremente esses graus. Reinsira-os só após escolher todos os d vizinhos. Se faltarem vizinhos de grau positivo, a sequência é impossível; terminar com todos os graus zero produz uma realização. Com heap, $O((n+m)\log n)$.

Teorema do casamento de Hall. Um grafo bipartido (L,R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \geq |S|.$$

Arestas possíveis e obrigatórias em matching perfeito. Fixe um matching *perfeito* M de um grafo bipartido simples (L,R). Oriente arestas fora de M de L para R, e as de M de R para L. Uma aresta fora de M pertence a algum matching perfeito sse seus extremos estão na mesma SCC. Uma aresta de M pertence a todos os matchings perfeitos sse seus extremos estão em SCCs distintas. Classifica todas as arestas em $O(V+E)$ após obter M.

Vértices que podem ficar sem par. Fixe um matching *máximo* de um grafo bipartido (L, R). Oriente arestas fora dele de L para R, e as dele de R para L. Um vértice de L fica livre em algum matching máximo sse é alcançável a partir dos livres de L. Para R, parta dos livres de R no grafo reverso. Inclua as origens. Duas buscas classificam todos em $O(V + E)$ após obter o matching; ser livre individualmente não implica poder liberar vários simultaneamente.

Teorema de König. Em um grafo bipartido, o máximo número de arestas sem extremos em comum (matching) é igual ao mínimo número de vértices que tocam todas as arestas (cobertura de vértices).

Para construir uma cobertura mínima C, sejam L, R as duas partes e fixe um matching máximo. Parta dos vértices não pareados de L; percorra arestas fora do matching de L para R e do matching de R para L. Se Z é o conjunto de vértices alcançados, $C = (L \setminus Z) \cup (R \cap Z)$. Os vértices fora de C formam um conjunto independente máximo.

Condições para trilha/circuito euleriano (direcionado). Desconsidere isolados; exija conectividade fraca:

- existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice;
- existe trilha aberta sse exatamente um vértice tem $\deg^+(v) = \deg^-(v)+1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.

Condições para trilha/circuito euleriano (não direcionado). Desconsidere isolados; exija conectividade:

- existe circuito euleriano sse todo vértice tem grau par;
- existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.

Restrições de diferença ↔ menores caminhos. Um sistema de desigualdades da forma

$$x_v - x_u \le w$$

pode ser modelado por uma aresta $u \to v$ de peso w. Então:

- o sistema é viável sse não existe ciclo negativo;
- rode Bellman–Ford com distâncias iniciais zero em todos os V vértices; as distâncias finais dão x_v. Relaxamento na V-ésima rodada indica ciclo negativo.

Cobertura por caminhos em DAG via matching. O número mínimo de caminhos disjuntos em vértices que cobrem todos os vértices de um DAG é

$$n - |M_{\max}|.$$

Crie cópias L, R dos vértices e aresta $u_L v_R$ para cada $u \to v$ original. Cada par do matching define o sucessor de u. Reconstrua começando nos vértices não pareados em R; inclua caminhos de um único vértice. Não use fecho transitivo para exigir caminhos disjuntos no grafo original.

Todos os cortes mínimos. Após um fluxo máximo s–t, considere apenas arcos de capacidade residual positiva. $(S, V \setminus S)$ é corte mínimo sse $s \in S$, $t \notin S$ e nenhum arco residual sai de S. Logo, cada SCC fica inteira em um lado; na condensação, incluir uma componente em S obriga incluir todas as alcançáveis. Vértices alcançáveis de s ficam sempre em S; os que alcançam t ficam sempre fora. O corte é único sse esses dois conjuntos cobrem todos os vértices.

Seleção com dependências (fecho de peso máximo). Cada item v tem ganho w_v (negativo para custo); escolher u pode obrigar escolher v. Maximize a soma dos ganhos respeitando todas as dependências. Crie $s \to v$ de capacidade w_v se positivo, $v \to t$ de capacidade $-w_v$ se negativo e $u \to v$ de capacidade INF para cada dependência. Use $\text{INF} > \sum_{w_v>0} w_v$. O ganho ótimo é essa soma menos o corte mínimo. Após fluxo máximo, escolha os itens alcançáveis de s no residual; escolher nenhum é permitido.

Custos binários por corte mínimo. Minimize $\sum D_v(x_v) + \sum E_{uv}(x_u, x_v)$, $x_v \in \{0, 1\}$; lado da fonte representa 0. Inicie constante $K = \sum D_v(0)$ e coeficientes $a_v = D_v(1) - D_v(0)$. Para cada par, exija $w = E_{01} + E_{10} - E_{00} - E_{11} \ge 0$; adicione $u \to v$ com capacidade w, some E_{00} a K, $E_{10} - E_{00}$ a a_u e $E_{11} - E_{10}$ a a_v. Depois: se $a_v \ge 0$, adicione $s \to v$ com capacidade a_v; senão, $v \to t$ com capacidade $-a_v$ e some a_v a K. Ótimo: $K+$ corte mínimo; recupere os rótulos pelos lados do corte.

Circulação com limites inferiores. Para $0 \le \ell_e \le f_e \le u_e$, use capacidade $u_e - \ell_e$ e $b(v) = \sum_{e \to v} \ell_e - \sum_{v \to e} \ell_e$. Adicione $S \to v$ com $b(v)$ se positivo, ou $v \to T$ com $-b(v)$ se negativo. Viável sse o fluxo $S \to T$ satura todas as arestas de S; recupere $f_e = \ell_e + f'_e$. Para fluxo $s \to t$ não negativo, adicione antes $t \to s$ com capacidade suficiente (por exemplo, $\sum u_e$).

Teorema Matrix-Tree de Kirchhoff. Em um grafo não direcionado, construa L inicialmente zero. Para cada aresta $u \ne v$, some 1 a L_{uu}, L_{vv} e subtraia 1 de L_{uv}, L_{vu}; arestas paralelas contam separadamente, self-loops são ignorados. Remova a mesma linha e coluna: o determinante da matriz restante é o número de árvores geradoras. Um único vértice tem uma árvore.

Kirchhoff direcionado. Para contar árvores em que todo vértice alcança a raiz r, inicie $L = 0$; cada arco $u \to v$, $u \ne v$, soma 1 a L_{uu} e subtrai 1 de L_{uv}. Remova linha e coluna r: $\det L_{\widehat{r},\widehat{r}}$ é a contagem. Para árvores alcançáveis a partir da raiz, inverta os arcos. Paralelos contam separadamente; pesos no lugar de 1 dão a soma dos produtos dos pesos das árvores.

BEST: contar circuitos eulerianos direcionados. Em um digrafo com arestas, graus de entrada e saída iguais e vértices ativos conexos no grafo subjacente, fixe a primeira aresta saindo de r. O número de circuitos é

$$t_r \prod_{v:\, d^+(v)>0} (d^+(v) - 1)!,$$

onde t_r conta árvores direcionadas para r no subgrafo ativo. Arestas paralelas são distintas; loops entram nos graus. Para fixar apenas o vértice inicial r, multiplique por $d^+(r)$. A fórmula com primeira aresta fixa também conta circuitos identificados por rotação.

Lindström–Gessel–Viennot (LGV). Em um DAG, sejam $a_1, \ldots, a_k$ origens distintas e $b_1, \ldots, b_k$ destinos distintos; M_{ij} conta caminhos de a_i a b_j. Então

$$\det M = \sum_{\pi \in S_k} \operatorname{sgn}(\pi) N_\pi,$$

onde N_π conta famílias de caminhos $a_i \to b_{\pi(i)}$ sem compartilhar vértices. Se apenas o pareamento $a_i \to b_i$ admite famílias disjuntas, $\det M$ é sua contagem; sem essa condição, é uma soma com sinais. Versão ponderada: M_{ij} soma produtos dos pesos das arestas dos caminhos; famílias recebem o produto dos pesos de seus caminhos.

Contar árvores geradoras mínimas. Processe pesos em ordem crescente, como Kruskal. Para cada peso, contraia as componentes formadas por pesos menores; descarte self-loops e preserve arestas paralelas. Em cada componente conexa do multigrafo desse peso, conte árvores geradoras por Kirchhoff e multiplique as contagens. Só depois una no DSU todas as arestas do grupo. Se o grafo original não ficar conexo, a resposta é zero.

Jogo UGV. Considere o jogo de mover ao longo de arestas de um grafo sem poder repetir vértices, e o jogador que não puder mover perde. O segundo jogador tem uma estratégia vencedora sse existe um matching máximo no grafo que deixa o vértice inicial insaturado.

Strings

Matching com wildcards por convolução. Padrão P de tamanho $m \geq 1$, texto T de tamanho $n \geq m$; codifique letras distintas por $1, \ldots, \sigma$ e wildcard por 0, em ambos. Cada wildcard substitui *um* caractere. Para $0 \leq s \leq n-m$,

$$D_s = \sum_{i=0}^{m-1} P_i T_{s+i} (P_i - T_{s+i})^2;$$

há match sse $D_s = 0$ nos inteiros. Para $r = 1, 2, 3$, tome $A_r = \mathrm{rev}(P^r)$ e $B_r = T^r$, com potências por elemento. Três convoluções lineares dão

$$C = A_3 * B_1 - 2(A_2 * B_2) + A_1 * B_3, \quad D_s = C_{m-1+s}.$$

Custo $O((n+m)\log(n+m))$ por módulo. Para NTT determinística, use primos distintos cujo produto seja $> m\sigma^2(\sigma-1)^2$ e exija zero em todos: esse é um limite superior de $D_s \geq 0$. Um módulo insuficiente pode aceitar falsos matches.

Teorema de Fine–Wilf. Se uma string tem períodos p e q, e seu tamanho é pelo menos

$$p + q - \gcd(p, q),$$

então ela também tem período $\gcd(p, q)$.

Substrings distintas. Usando array de sufixos:

$$\frac{n(n+1)}{2} - \sum lcp[i]$$

Usando autômato de sufixos:

$$\sum_{i \neq \text{raiz}} \big(\texttt{len}(i) - \texttt{len}(\texttt{link}(i))\big)$$

Notas de autômato de sufixos. O automato de sufixos é um DAG. Cada caminho a partir da raiz representa uma substring. Cada nó representa um conjunto de `endpos`. `link(i)` é o maior sufixo com um conjunto `endpos` diferente.

Encontrando a maior substring comum. Construa o automato de sufixos de uma das strings. Itere pela segunda string mantendo o maior sufixo dela que aparece na primeira string. Quando uma transição falha, basta subir por suffix links, e o novo maior tamanho do sufixo vira `len(v)`.

Geometria

Teorema de Euler para grafos planares. Num grafo planar com v vértices, e arestas, f faces e k componentes conexas,

$$v - e + f = 1 + k$$

Teorema de Pick. Para um polígono simples na grid,

$$A = I + \frac{B}{2} - 1,$$

onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.

Fórmula de Heron. Para lados a, b, c e semiperímetro $s = (a+b+c)/2$, a área é

$$\sqrt{s(s-a)(s-b)(s-c)}.$$

Raio da circunferência circunscrita e inscrita. Para área do triângulo Δ:

$$R = \frac{abc}{4\Delta}, \qquad r = \frac{\Delta}{s}$$

onde $s = \frac{a+b+c}{2}$.

Teorema de Helly em $\mathbb{R}^d$. Para uma família finita de convexos em $\mathbb{R}^d$, se toda subfamília de até $d+1$ conjuntos tem interseção não vazia, a família inteira também tem.

Debugging

Confira cada grupo mesmo quando parecer correto. Reproduza a falha e reduza o caso; compare estados intermediários com uma simulação manual ou força bruta.

Enunciado e modelo

- ☐ Releia o que deve ser contado/otimizado: ordem importa? Repetições são distintas? A escolha vazia é permitida?
- ☐ Confira as hipóteses do algoritmo: pesos negativos, monotonicidade, conectividade, empates. Elas vêm do enunciado ou foram presumidas?
- ☐ Siga à mão um caso pequeno; confira o significado de cada estado, transição, base e resposta extraída.

Limites e índices

- ☐ Force menor tamanho permitido, um elemento, todos iguais, valores extremos e resposta inexistente.
- ☐ Confira cada acesso: 0/1-based, n/m trocados, $i-1$, $i+1$, sentinelas $-1/n$, tamanho $n+1/2n$.
- ☐ Refaça os extremos dos intervalos: $[l, r)$ ou $[l, r]$? Teste vazio, unitário e intervalo inteiro; confira $<$ versus $\leq$.

Aritmética

- ☐ Verifique o tipo de cada produto, soma e shift *antes* da atribuição; cabe no tipo? Alguma operação usa `INF`?
- ☐ Confira módulo, resíduos negativos e divisão modular (o inverso existe?). Na divisão inteira, o arredondamento é o desejado?
- ☐ Teste denominador zero, raiz de negativo, `NaN`/infinito e cancelamento numérico; confira tolerância e precisão da saída.

Estado e atualizações

- ☐ Rode o mesmo caso sozinho e após outro: globais, containers, flags e lazy tags foram reinicializados?
- ☐ Cada valor foi inicializado antes de ser lido? Algum estado impossível foi usado como válido?
- ☐ A atualização usa o estado anterior ou já alterado? Confira direção do loop, ordem de composição e rollback; referências/iteradores continuam válidos?

Representação e ordem

- ☐ Teste componentes desconexas, isolados, self-loops e arestas paralelas; direção, aresta reversa e identificação pelo pai estão corretas?
- ☐ Force empates: a ordenação tem desempate correto e comparador estrito? Eventos na mesma coordenada entram/saem na ordem certa?
- ☐ Após ordenar/comprimir, os IDs e pesos ainda correspondem? A compressão perdeu distâncias necessárias? A reconstrução produz uma solução válida?

Tempo e memória

- ☐ Cada loop progride? Recursão revisita estados ou excede a pilha? Teste cadeia, estrela e entrada adversária para a estrutura.
- ☐ Conte cópias, alocações, limpeza por teste e fatores extras de $\log n$; confira o custo total de todos os casos e o argumento amortizado.
- ☐ Calcule o tamanho real dos buffers/estados; confira se containers crescem sem limite ou se estruturas locais grandes ocupam a pilha.

Execução e saída

- ☐ Recompile o arquivo correto e rode com sanitizers. Leia todos os campos e casos? Removeu prints de debug? Formato e precisão estão corretos?
- ☐ Em stress testing, o gerador respeita as restrições e a força bruta é independente? Respostas múltiplas exigem validar a solução, não igualdade textual.